This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Neural Combinatorial Optimization Algorithms for Solving Vehicle Routing Problems: A Comprehensive Survey With Perspectives

Xuan Wu, Lijie Wen, Yubin Xiao, Chunguo Wu, Yuesong Wu, Chaoyu Yu, Douglas L. Maskell, *Life Senior Member, IEEE*, You Zhou, and Di Wang, *Senior Member, IEEE*

***Abstract*—Although several surveys on neural combinatorial optimization (NCO) solvers specifically designed to solve vehicle routing problems (VRPs) have been conducted, they did not cover the state-of-the-art (SOTA) NCO solvers emerged recently. More importantly, to establish a comprehensive and up-to-date taxonomy of NCO solvers, we systematically review relevant publications and preprints, categorizing them into four distinct types, namely learning to construct (L2C), learning to improve (L2I), learning to predict (L2P)-once, and L2P-multiplicity (L2P-M) solvers. Subsequently, we present the inadequacies of the SOTA solvers, including poor generalization, incapability to solve large-scale VRPs, inability to address most types of VRP variants simultaneously, and difficulty in comparing these NCO solvers with the conventional operations research (OR) algorithms. Simultaneously, we discuss ongoing efforts, identify open inadequacies, and propose promising and viable directions to overcome these inadequacies. Notably, existing efforts focus on only one or two of these inadequacies, with none attempting to address all of them concurrently. In addition, we compare the performance of representative NCO solvers from the reinforcement, supervised, and unsupervised learning (UL) paradigms across VRPs of varying scales. Finally, following the proposed taxonomy, we provide an accompanying web page as a live repository for NCO solvers. Through this survey and the live repository, we aim to foster further advancements in the NCO community.**

***Index Terms*—Data-driven optimization, neural combinatorial optimization (NCO), vehicle routing problem (VRP).**

I. Introduction

COMBINATORIAL optimization problem (COP) is a vital branch of mathematical optimization, dedicated to seeking optimal solutions for problems within the discrete space [1], [2], [3]. Among various COPs, vehicle routing problems (VRPs) have extensive practical applications across diverse domains, e.g., communication and transportation [4], [5], [6], [7]. Generally speaking, operations research (OR) algorithms for solving VRPs can be broadly divided into three categories, namely exact, approximation, and heuristic algorithms [8]. Specifically, exact algorithms typically employ the divide-and-conquer (D&C) manner to solve VRPs and obtain optimal solutions. These algorithms mainly encompass branch and bound [9] and dynamic programming (DP) [10]. Approximation algorithms, aimed at securing solutions of assured quality, include relaxation algorithm [11] and linear programming [12]. Heuristic algorithms represent a class of methods that utilize predefined heuristic rules to efficiently explore solutions in a given timeframe, developed through continuous trial-and-error. The most prominent heuristic algorithm for solving VRPs is Lin–Kernighan–Helsgaun 3 (LKH3) [13]. In addition, it is noteworthy that approximation and heuristic algorithms may have the potential to converge to the optimal solutions given sufficient time, yet such convergence is not guaranteed [8], [14], [15].

However, these three types of conventional OR algorithms lack the ability to derive insights from historical VRP instances, leading to significant computing overhead, especially for medium- or large-scale VRPs [16], [17]. This is demonstrated by the performance of the LKH3 algorithm in solving a 100-node capacitated VRP (CVRP) instance, requiring a noteworthy 12 h to attain the optimal solution [18]. This shortcoming constrains the applications of OR algorithms in time-sensitive scenarios, such as on-call routing [19] and ride-hailing service [20]. To alleviate the expensive computing overhead, many neural combinatorial optimization (NCO) solvers utilizing deep learning (DL) have recently emerged [21], [22], [23], [24], [25], [26], [27], [28], as shown in Fig. 1. These NCO solvers possess the ability to learn from extensive

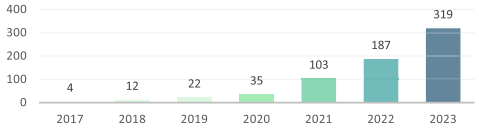


Fig. 1. Illustration of the number of publications and preprints on NCO solvers for VRPs. This information is gathered from Google Scholar and Web of Science with the keywords "neural combinatorial optimization" *OR* "NCO" *OR* "reinforcement learning" *OR* "deep learning" *OR* "neural network" *AND* "vehicle routing problem" *OR* "VRP" *OR* "traveling salesman problem" *OR* "TSP" by the end of 2023. Following the initial data collection, a meticulous examination of each literature piece is conducted to precisely define its scope within the realm of NCO.

Received 28 July 2024; revised 25 April 2025, 24 August 2025, 23 November 2025, and 9 March 2026; accepted 7 July 2026. This work was supported by Jilin Provincial Department of Science and Technology Project under Grant 20240101369JC. *(Corresponding authors: Yubin Xiao; You Zhou.)*

Xuan Wu, Yubin Xiao, Chunguo Wu, Yuesong Wu, Chaoyu Yu, and You Zhou are with the Key Laboratory of Symbolic Computation and Knowledge Engineering of the Ministry of Education, College of Computer Science and Technology, Jilin University, Changchun 130012, China (e-mail: wuuu22@mails.jlu.edu.cn; xiaoyb21@mails.jlu.edu.cn; wucg@jlu.edu.cn zhaoyuy22@mails.jlu.edu.cn; wucg@jlu.edu.cn; zyou@jlu.edu.cn).

Lijie Wen is with the School of Software, Tsinghua University, Beijing 100084, China (e-mail: wenlj@tsinghua.edu.cn).

Douglas L. Maskell is with the College of Computing and Data Science, Nanyang Technological University, Singapore 639798 (e-mail: asdouglas@ntu.edu.sg).

Di Wang is with the Joint NTU-UBC Research Center of Excellence in Active Living for the Elderly, Nanyang Technological University, Singapore 639798 (e-mail: wangdi@ntu.edu.sg).

Digital Object Identifier 10.1109/TNNLS.2026.3713193

2162-237X © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:48:06 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

historical instances and efficiently search for (sub)optimal solutions when dealing with instances exhibiting the same distribution characteristics. Moreover, by leveraging GPUs, these NCO solvers demonstrate efficiency in concurrently handling batches of instances, thereby further enhancing the speed of problem-solving [29], [30], [31], [32]. For example, for a 100-node CVRP instance, an NCO solver named POMO [18] requires only one minute to find the suboptimal solution with a gap value of less than 1%. Here, the gap denotes the relative difference in the "average length" between POMO and LKH3.

In the past three years, a number of surveys [8], [21], [22], [29], [33], [34], [35], [36], [37], [38], [39], [40], [41], [42] have been conducted to introduce and summarize NCO solvers specifically designed for VRPs. Nonetheless, these existing surveys have various limitations. First, the taxonomies proposed in these surveys [8], [21], [29], [33], [34] do not facilitate an appropriate category for all NCO solvers, due to the rapid emergence of numerous novel solvers. Secondly, a number of surveys [8], [21], [22], [33], [34], [35], [36], [37], [38], [39] do not adequately cover current NCO solvers' inadequacies and associated efforts, thereby falling short in providing promising research directions. For example, a recent survey [42] did not discuss the inadequacy of existing solvers in handling large-scale instances. Thirdly, certain surveys [35], [36], [37], including the most recent ones [40], [41], exclusively focus on NCO solvers utilizing the reinforcement learning (RL) paradigm, overlooking those based on supervised learning (SL) and unsupervised learning (UL). Although RL remains the most commonly adopted paradigm, certain state-of-the-art (SOTA) solvers have been proposed under SL or UL paradigms. Consequently, these surveys do not provide a comprehensive overview of the current advances in the field. Finally, many surveys [8], [21], [29], [34], [35], [36] focus on multiple COPs, without providing a detailed introduction to VRPs. All these necessitate a more up-to-date survey to include the SOTA advancements in this important research field. In the following paragraphs, we elaborate on these four limitations of existing surveys and introduce the advantages of this survey.

Regarding the first limitation, the two taxonomies proposed in prior surveys [8], [21], [29], [33], [34] fall short in appropriately classifying all NCO solvers. Specifically, based on the adopted learning paradigm, the first taxonomy [29] classifies NCO solvers into three categories, namely solvers adopting constructive deep RL, improving deep RL, and SL or RL. However, the solving process of SL solvers (e.g., [31], [43], [44]) aligns with solvers adopting constructive deep RL (e.g., [18], [32]). In essence, either type of solvers [18], [31], [43], or [32] and [44] employ the learned heuristics to select the unvisited nodes and add them to partial solutions sequentially. Consequently, it is not appropriate to classify NCO solvers solely based on the adopted learning paradigm. The other taxonomy categorizes NCO solvers into three types, namely end-to-end (EE), learning to configure (LC), and multiple calls (MCs) solvers [21]. EE solvers employ a neural network (NN) dedicated to solving VRPs in isolation, LC solvers offer the parameters of OR algorithms (only once), and MC solvers assist OR algorithms in making lower level decisions multiple times throughout the search process. However, certain NCO solvers do not fit into any of the categories in the second taxonomy. For example, Joshi et al. [45] predicted key information only once in the whole search process of Beam Search, diverging from providing parameters for OR algorithms, thereby placing this solver outside the categories proposed in [21]. Therefore, the second taxonomy has its drawbacks in categorizing all NCO solvers.

TABLE I
OUR PROPOSED TAXONOMY OF NCO SOLVERS

Category	Description	Representatives
L2C	Using NNs to construct solutions from scratch	Ptr-Net [31]; AM [32]; POMO [18]; NAR4TSP [46]
L2I	Using NNs to iteratively improve complete solutions	NeuRewriter [47]; MT [48]; N2OPT [49]; NeuOPT [50]
L2P-O	Using NNs to assist OR algorithms once	GCN [45]; NeuroLKH [51]; DPDP [52]; DIFUSCO [53]
L2P-M	Using NNs to assist OR algorithms multiplicity	GAVE [54]; VSR-LKH [55]; MOCO [56]; EOH [57]

In this survey, we propose a novel taxonomy aimed at appropriately and exclusively classifying all NCO solvers. As shown in Table I, we categorize these solvers into four categories, namely learning to construct (L2C), learning to improve (L2I), learning to predict (L2P)-once (L2P-O), and L2P-multiplicity (L2P-M) solvers. L2C solvers represent the pioneering category of NCO solvers, drawing inspiration from the analogy between VRPs and the machine translation task. Specifically, L2C solvers use NNs to sequentially select nodes that have yet to be visited (resembling the process of translating words in the machine translation task) and add them to partial solutions [32], [58], [59]. For example, Vinyals et al. [31] were the pioneers who employed SL to train a recurrent NN (RNN) for sequentially constructing solutions for the traveling salesman problem (TSP). Subsequently, inspired by the notion of iteratively ruining and repairing in heuristic algorithms [60], certain L2I solvers [6], [47], [61], [62] introduce NNs to emulate this process, aiming to derive the (sub)optimal solutions. For example, Chen and Tian [47] selectively deconstructed the local components of complete solutions and repaired them through a learned policy, enhancing the quality of current complete solutions. Meanwhile, certain L2P-O solvers [45], [52] and L2P-M solvers [54], [55] diverge from emulating the iteratively improving process. Instead, these solvers focus on integrating the predicted key information with OR algorithms to enhance solution quality and/or reduce search overhead. Furthermore, L2P-O solvers predict information only once throughout the entire search process of OR algorithms, whereas L2P-M ones require the facilitation of OR algorithms during the whole search process, making predictions and decisions multiple times. For example, to expedite the search process of DP, Kool et al. [52] proposed an L2P-O solver named deep policy DP (DPDP). This solver predicts promising edges for each TSP instance before the initiation of the DP search. The solution is then exclusively constructed using these identified promising edges during the subsequent DP search. In line with the objective of [52] to expedite the search process of OR algorithms, an L2P-M solver named VSR-LKH [55] exploits RL to inform decisions at each search step of the LKH algorithm [63]. In conclusion, the taxonomy proposed in this article provides a structured framework to comprehensively present the diversity of NCO solvers, ensuring that all solvers can be exclusively categorized into one of the four categories defined in our taxonomy. In Sections III–V, we discuss the reasonableness and exclusivity of the proposed taxonomy, providing comprehensive overviews of the relevant studies corresponding to each solver category.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:48:06 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

The second limitation arises from the lack of comprehensive reporting on the inadequacies of SOTA NCO solvers in prior surveys [8], [21], [22], [34], [35], [36], [37], [64]. Unfortunately, they only introduce the relatively outdated inadequacies of NCO solvers, failing to provide researchers and practitioners with insights into the latest developments in NCO solvers. For example, certain surveys identify the performance of NCO solvers on small- or medium-scale VRPs as a potential inadequacy. However, recent NCO solvers (e.g., GELD [65]) have effectively addressed this inadequacy, achieving performance on par with exact algorithms. Again, a more up-to-date survey is required to reflect these recent advancements. To identify the open inadequacies of SOTA NCO solvers, we conduct a systematic survey and find that these inadequacies primarily stem from intrinsic requirements of VRPs in practical applications. It is worth highlighting that we do not focus on the limitations of individual or a small group of solvers in this work. Instead, we examine pervasive inadequacies faced by the majority of SOTA solvers across one or more VRP variants, which stem from the intrinsic VRP requirements or the nature of the real-world VRP applications, such as generalization across different scales. Sections VI-A–VI-D introduce these four open inadequacies in detail and simultaneously introduce ongoing efforts to address them, respectively. Specifically, the first inadequacy is that existing NCO solvers are only capable of solving VRPs that have the same instance distribution [66], [67], [68], [69]. For example, NCO solvers trained on a type of specific instance distribution (e.g., the uniform distribution) might not perform well when being applied to instances from other distribution patterns (e.g., the cluster distribution) [67]. In addition, existing NCO solvers face inadequacies in addressing large-scale VRPs (e.g., the 10 000-node TSP) in real-time [70], [71], [72]. Moreover, existing NCO solvers exhibit limited performance on various VRP variants (e.g., asymmetric TSP [73]), necessitating the development of specific NNs for each VRP variant. Finally, the issue of unfair comparisons emerges as a substantial inadequacy in evaluating NCO solvers against the conventional OR algorithms, resulting in a constrained acceptance of these NCO solvers within the OR community [74], [75]. To better solve these inadequacies, recent studies [65], [76], [77] have explored various approaches and proposed corresponding solvers, which have been reviewed in this survey. More importantly, apart from introducing ongoing efforts to address these inadequacies, we also identify open inadequacies and suggest promising research directions to address them (see Section VI for more details), which are not dedicatedly and comprehensively discussed in existing surveys. It is worth highlighting that while certain prior studies (e.g., GELD [65]) have attempted to tackle one or two of these inadequacies, none simultaneously addresses three or even all four inadequacies, and the overall performance of many NCO solvers still leaves room for improvement (see Section VI-C for more details).

The third limitation stems from the fact that certain surveys [35], [36], [37], [78] exclusively focus on NCO solvers utilizing RL, overlooking those based on SL and UL. Consequently, these surveys do not provide a comprehensive performance comparison across all solvers, which is a non-negligible issue due to the fact that certain SOTA SL- and UL-based solvers [53], [79] have demonstrated high-level performance on large-scale VRPs. In contrast, our survey stands out by introducing NCO solvers based on all learning paradigms, namely SL, UL, and RL. For example, we introduce the SL-based solver LEHD [44] and the UL-based solver UTSP [79], both of which were specifically designed to solve large-scale VRPs by reducing the number of parameters (see Section VI-B for more details). By employing this comprehensive survey methodology, this survey paper provides a thorough performance comparison among solvers across varying scales of VRPs, which is largely absent from existing surveys.

Finally, a number of existing surveys [8], [21], [29], [34], [35], [36] have delved into a spectrum of COPs, such as job shop scheduling problem (JSP) [80], [81], [82], [83] and knapsack problem (KP) [84], [85], [86]. Consequently, these surveys are unable to provide a comprehensive and detailed introduction to VRPs. In contrast, this survey paper takes a focused approach to comprehensively introduce the details of NCO solvers tailored for VRPs, the most researched COPs in the NCO domain [87]. For example, in Section III-D, we elaborate on data augmentation methods specially designed for VRPs, which are often overlooked in other surveys. Moreover, another reason for the exclusive focus of this survey on VRPs is that most NCO studies initially focused on VRPs before extending their applications to other COPs. For example, Kool et al. [32] pioneered the integration of the attention model (AM) into NCO to solve VRPs, which is expanded upon by subsequent studies [82], [85] to tackle JSPs and KPs, respectively. Hence, it is justified for us to purposely focus on VRPs in this survey.

The key contributions of this work are as follows.

1) This survey paper proposes a novel taxonomy for NCO solvers specifically designed to solve VRPs. Specifically, the proposed taxonomy categorizes existing NCO solvers into four exclusive types.
2) This article outlines the inadequacies of existing NCO solvers, including generalization, large-scale and multiconstrained VRPs, and fair comparisons with OR algorithms, while introducing ongoing efforts and suggesting promising directions to better address these inadequacies.
3) This article explores NCO solvers across diverse learning paradigms for VRPs, introduces them in a detailed manner, and comprehensively assesses their performance across different problem scales.
4) This article provides an accompanying web page[1] as a live repository of papers on NCO solvers organized according to our proposed taxonomy. In addition, this web page will be continuously updated with emerging studies.

II. Preliminary

Section II-A first introduces the definitions of TSP and CVRP. Subsequently, Section II-B presents three learning paradigms of DL, namely SL, UL, and RL.

A. Definitions of TSP and CVRP

In this section, we specifically focus on two kinds of VRP variants, namely TSP and CVRP, due to their importance and prevalence [32]. For TSP, let $G(\boldsymbol{V}, \boldsymbol{E})$ denote the graph of a VRP with N cities, where $\boldsymbol{V}$ and $\boldsymbol{E}$ denote the vertex (i.e., $|\boldsymbol{V}| = N$) and edge sets, respectively. The edge $e_{i,j} \in \boldsymbol{E}$ denotes the Euclidean distance between the ith node v_i and jth node v_j. In addition, $x_{i,j} = 1$ indicates that v_j is visited

[1]URL: https://github.com/wuuu110/NCO-solvers-for-VRP

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:48:06 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

directly after v_i. Conversely, $x_{i,j} = 0$ indicates that v_j is not visited directly after v_i. Finally, let τ_i denote the visiting order of the ith node. Therefore, the Miller–Tucker–Zemlin formulation [88] of TSP is given as follows:

$$\min \sum_{i=1}^{N} \sum_{j=1, j\neq i}^{N} e_{i,j} \cdot x_{i,j}, \quad x_{i,j} \in \{0,1\} \tag{1}$$

$$\text{s.t.} \sum_{i=1, i\neq j}^{N} x_{i,j} = 1 \quad \forall j \in \{1,2,\ldots,N\} \tag{2}$$

$$\sum_{j=1, j\neq i}^{N} x_{i,j} = 1 \quad \forall i \in \{1,2,\ldots,N\} \tag{3}$$

$$\tau_i - \tau_j + N \cdot x_{i,j} \leq N-1, \quad 1 \leq i \neq j \leq N. \tag{4}$$

For TSP, the objective is to minimize the total distance cost of the tour (solution) computed according to (1). Constraints (2) and (3) ensure that each node can be visited only once. Finally, constraint (4) eliminates subtours.

Compared to TSP, CVRP has the same objective, i.e., minimizing the total distances of the tour. But CVRP has more constraints due to varying goods demands in each customer and a maximum capacity limit for the delivery vehicle (assuming only one vehicle is available for all CVRP instances in this article). Constrained by the capacity constraint, the vehicle visits partial nodes and subsequently returns to the depot; this subtour is termed a route. Specifically, let the 0th node and d_i denote the depot node and the demand of the ith customer node, respectively. In addition, let $e_{i,j}$, u_i, and C denote the distance cost of a vehicle going from ith node to jth node, where $i, j \in \{0,1,2,\ldots,N\}$, the surplus capacity of the vehicle after it serves the ith customer node, and the maximum capacity of a vehicle, respectively. An integer programming formulation of CVRP [11] is given as follows:

$$\min \sum_{i=0}^{N} \sum_{j=0}^{N} e_{i,j} \cdot x_{i,j}, \quad x_{i,j} \in \{0,1\} \tag{5}$$

$$\text{s.t.} \sum_{i=0, i\neq j}^{N} x_{i,j} = 1 \quad \forall j \in \{1,2,\ldots,N\} \tag{6}$$

$$\sum_{j=0, j\neq i}^{N} x_{i,j} = 1 \quad \forall i \in \{1,2,\ldots,N\} \tag{7}$$

$$u_j = \begin{cases} u_i - d_j, & x_{i,j} = 1, u_i > d_j, \ j \neq 0 \\ C, & x_{i,j} = 1, \quad j = 0 \end{cases} \tag{8}$$

$$0 \leq d_i, \quad u_i \leq C \quad \forall i \in \{1,2,\ldots,N\}. \tag{9}$$

For CVRP, constraints (6) and (7) ensure that each node can be visited only once, apart from the depot node. Finally, constraints (8) and (9) impose the vehicle capacity requirements.

The objectives of other VRP variants may be subject to different problem-specific constraints. For example, in the case of VRP with time window (VRPTW), node visits must adhere to specified time windows. In Table II, apart from TSP and CVRP, we also present four widely studied VRP variants, namely VRPTW, multiobjective VRP (MOVRP), dynamic VRP (DVRP), and pickup-and-delivery VRP (PDVRP). As indicated by the little overlap on the references listed in Table II, most solvers cannot simultaneously address all or most types of VRP variants. Therefore, researchers must develop specialized NCO solvers to address VRP variants with additional constraints. See Section VI-C for in-depth insights into the design principles of NCO solvers for solving VRPs with more constraints.

TABLE II
DIFFERENT VRP VARIANTS AND CORRESPONDING NCO SOLVERS

Variant	Description	Corresponding Studies
TSP	All nodes are visited once, starting and ending with the same node.	[18], [31], [32], [48], [59], [70], [71], [89]–[103]
CVRP	The cumulative demands of visited customer nodes for a single route must not surpass the assigned vehicle capacity.	[18], [32], [47], [48], [59], [89]–[92], [95]–[98], [100], [101], [104]–[106]
VRPTW	Node visits must adhere to specified time windows. Vehicles can wait in the event of early arrival, but late arrival is not permissible.	[106]–[116]
MOVRP	In addition to minimizing tour length, additional objectives include minimizing traveling time, etc.	[117]–[127]
DVRP	Dynamically scheduling vehicles to achieve online-generated orders across multiple locations.	[128]–[135]
PDVRP	Visiting the pickup node prior to the associated delivery node in a service request.	[136]–[138]

B. Learning Paradigms of DL

In this section, we introduce three kinds of DL training paradigms, namely SL, UL, and RL, where RL is widely adopted by NCO solvers for training NNs [35], [36], [37].

1) Supervised Learning: SL is a paradigm in DL where the model undergoes training on a labeled dataset, containing input–output pairs. The objective of the model is to learn a function that produces predicted outputs closely matching the corresponding output labels, when given a set of input data. In the field of NCO, Vinyals et al. [31] were the first to use a supervised loss function for training an RNN. The loss is computed as follows:

$$loss = -\sum_{\mathcal{S},\tau^*} \log p_\theta(\tau^*|\mathcal{S}) \tag{10}$$

where θ, $\mathcal{S}$, and τ^* denote the parameters of the RNN model, an instance with a number of N nodes drawn from a specific type of distribution $\mathbb{S}$, and a permutation of integers representing the optimal tour of $\mathcal{S}$, respectively. Finally, $p_\theta(\tau^*|\mathcal{S})$ denotes the conditional probability of output τ^* given input $\mathcal{S}$, which is computed as follows:

$$p_\theta(\tau^*|\mathcal{S}) = \prod_{i=1}^{N} p\left(\tau_i^*|\tau_{(j,j<i)}^*, \mathcal{S}; \theta\right). \tag{11}$$

However, acquiring labels for VRP instances, especially for large-scale VRPs, is often impractical or time-intensive. Furthermore, models trained using SL tend to show poor generalization, because they tend to capture noise or specific patterns that fail to generalize to new and unseen data [92]. These issues limit the widespread adoption of SL in solving VRPs [18]. Recently, to reduce the computing overhead of obtaining labels, certain studies [43] proposed various data augmentation methods (see Section III-D for more details).

2) Reinforcement Learning: RL represents a distinct paradigm in DL, where agent(s) acquire decision-making abilities through interactions with the environment. As shown in Fig. 2, we describe the interacting process between a single agent and the environment of RL. At time t, the agent selects

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:48:06 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

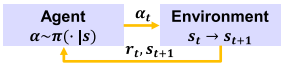


Fig. 2. Illustration of interactions between the agent and the environment in RL, where t, a_t, s_t, π, and r_t denote the time, action, state, policy, and reward, respectively. The policy $\pi(\cdot|s)$ of the agent takes the state s_t as input and selects the action a_t as output. Following this, the action is executed in the environment, and the agent receives a reward r_t. This sequential process continues as the state transitions to s_{t+1}.

an action a_t according to the state s_t. Subsequently, the state transitions to s_{t+1}, and the environment provides feedback in the form of a reward r_t for the agent.

Compared with SL-based solvers, RL ones autonomously learn to solve VRPs without human guidance and the effort of data labeling, thereby addressing the inherent limitation of SL's reliance on high-quality labels. In addition, RL-based solvers consistently surpass SL-based solvers in terms of generalization due to RL's interactive and exploratory nature [139]. Hence, following the study of [92], who introduced RL for training NNs to solve VRPs in 2017, a large number of RL-based solvers emerged [18], [32], [59], [90], [140], [141], [142], [143].

Specifically, the objective of the solver proposed in [92] is to maximize the expected return (total reward) J, which is given as follows:

$$J(\theta|\mathcal{S}) = -\mathbb{E}_{\tau\sim p_\theta(\cdot|\mathcal{S})} L(\tau|\mathcal{S}) \tag{12}$$

where $L(\cdot)$ produces the length of instance $\mathcal{S}$. The policy $p_\theta(\cdot|\mathcal{S})$ selects a solution τ given a VRP instance $\mathcal{S}$, which is factorized and parameterized by parameters θ as follows:

$$p_\theta(\tau|\mathcal{S}) = \prod_{i=1}^{N} p\left(\tau_i|\tau_{(j,j<i)}, \mathcal{S}; \theta\right) \tag{13}$$

where τ_i and $\tau_{(j,j<i)}$ denote the current action at time i and the previous actions, respectively. To optimize the parameters θ, Bello et al. [92] employed the well-known REINFORCE algorithm [144], which is a policy gradient method incorporating stochastic gradient descent to compute the gradient of (13). The gradient is computed as follows:

$$\nabla_\theta J(\theta|\mathcal{S}) = \mathbb{E}_{\tau\sim p_\theta(\cdot|\mathcal{S})}\left[\left(L(\tau|\mathcal{S}) - b(\mathcal{S})\right)\nabla_\theta \log p_\theta(\tau|\mathcal{S})\right]. \tag{14}$$

where $b(\mathcal{S})$ denotes the baseline function, which remains independent of the solution τ and serves to estimate the expected tour length, thereby reducing the variance of the gradients. Specifically, in [92], an exponential moving average of returns obtained by the policy network over time is employed as the baseline function $b(\mathcal{S})$.

It is worth mentioning that more effective baselines have been proposed in subsequent studies [18], [49], [90], [141]. One of the most straightforward yet most efficient baselines is the shared baseline proposed in POMO [18]. Specifically, in contrast to [92], where the NN consistently selects the fixed node as the starting node to generate a single solution for each instance, POMO mandates the generation of multiple solutions with diverse starting nodes for each instance. Subsequently, the averaged value of a set of solutions generated from different starting nodes is employed as the baseline of the advantage function. However, applying RL to train NCO solvers still faces a fundamental challenge. Specifically, as the policy improves, the magnitude of the advantage values diminishes. Because RL depends on these numerical signals to guide learning, their diminishing scale inevitably leads to vanishing gradients and slow convergence. A promising future research direction for RL-based NCO solvers would be solving this problem of learning signals becoming less significant in the later iterations.

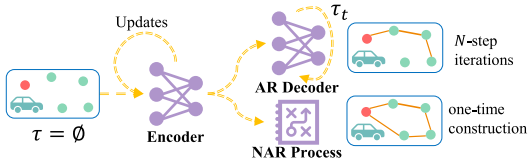


Fig. 3. Illustration of the generic construction process of L2C solvers, starting from an empty solution set and ending with complete solutions. Most L2C solvers are composed of an encoder and a decoder. The encoder is used to output the embeddings of VRP instances, while the decoder selects nodes based on these embeddings to construct complete solutions.

3) Unsupervised Learning: Unlike the other two learning paradigms, UL does not require explicitly (i.e., labels in SL) or implicitly (i.e., rewards in RL) given information for learning. Instead, it endeavors to capture certain characteristics of the observed random variables. Until now, the combination of UL and VRP has received little attention [21], [29]. Consequently, this survey refrains from an extensive exploration of UL, addressing it only when necessary. For instance, in Section VI-B, we present an L2P-O solver named UTSP [79], which incorporates a UL-based loss function to conserve memory and consequently address large-scale VRPs.

III. L2C Solvers

As shown in Fig. 3, similar to the machine translation task, L2C solvers start from an empty solution set, selecting unvisited nodes to construct complete solutions for VRPs. Generally speaking, L2C solvers are composed of an encoder and a decoder, predominantly leveraging a Markov decision process (MDP) formulation to learn how to solve VRPs. To offer a comprehensive review of L2C solvers, Sections III-A–III-C introduce the design of the encoder, the design of the decoding process, and the MDP of L2C, respectively. Subsequently, Section III-D presents data augmentation methods and postprocessing strategies. Finally, Section III-E compares the performance of L2C solvers on small-scale VRPs.

A. Design of Encoders

In the field of NCO, Bello et al. [92] were the first to employ actor–critic RL for training long short-term memory (LSTM) [145] as an encoder, capturing the coordinate information of nodes and converting it into embeddings. Subsequently, to fully leverage the graph structure, Khalil et al. [93] proposed a graph NN (GNN)-based S2V-DQN to embed node features. Differing from the previously employed RNN-based and GNN-based encoders, Kool et al. [32] proposed AM based on the superior transformer architecture [146]. Specifically, as shown in Fig. 4, Kool et al. [32] adopted a total of three attention layers as an encoder, with each layer comprising a multihead attention and a node-wise feed-forward sublayer.

Ever since, AM has become a prevalent encoder model choice, adopted by many L2C solvers [18], [59], [90], [143], [147], [148] as the backbone structure. For example, to pursue better performance, Kwon et al. [18] further extended the number of attention layers of AM from three to six. Furthermore, the following paragraphs present two primary

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:48:06 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

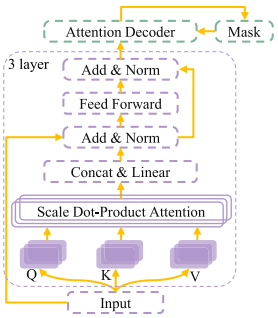


Fig. 4. Illustration of AM [32].

transformer-based encoder designs tailored for specific problem scales and VRPs.

The first design involves certain L2C solvers [44], [141], [149] propose innovative transformer-based encoders with the objective of reducing the number of parameters to solve large-scale VRPs. This design choice stems from the observation that with the increasing scale of VRP, transformer-based solvers experience a substantial surge in computation time and memory overhead. Consequently, L2C solvers endowed with a large number of parameters struggle to effectively address large-scale VRP instances [44]. To better solve this problem, Jin et al. [141] adopted the reversible residual network [150] instead of the standard residual network in the transformer-based encoder. A reversible residual network does not require a large number of parameters by eliminating the need to store activation values in most layers. These values can be accurately reconstructed based on the activation values of the subsequent layer. Meanwhile, Yang et al. [149] replaced scaled dot-product attention with sampled scaled dot-product attention [151], where only the query and key of a small number of sampled nodes are computed rather than those of all nodes. While these solvers do reduce the number of parameters to some extent, their effectiveness in addressing large-scale VRPs still needs improvement, compared with other solvers (see Section VI-B for more details).

Meanwhile, as shown in Fig. 3, certain L2C solvers opt for the second design, i.e., real-time embedding updates. This design stems from the fact that many L2C solvers do not dynamically update node embeddings throughout the decoding process, potentially hindering their ability to model the dynamics of state transitions [94], [97]. This limitation is particularly noticeable for VRPs characterized by dynamic patterns, e.g., the covering salesman problem [152]. To solve this problem, Xin et al. [95] proposed a dynamic embedding method based on AM, which explicitly eliminates the visited nodes and extracts the updated feature embeddings in each decoding step. Similarly, Li et al. [153] proposed a transformer-based NN to adaptively update the node embeddings with the guidance of the historically constructed solutions. This type of encoder design usually strives for better performance by leveraging more time and computing resources.

In summary, the superior performance of AM has established it as the predominant encoder type. While certain studies considered the demands of specific VRP instances, thus tailoring the transformer-based encoder design accordingly.

B. Design of Decoding Processes

The decoder of most L2C solvers adopts the autoregressive (AR) approach, selecting a node on the Hamiltonian loop at each decoding step based on the embeddings produced by the encoder. Moreover, the encoders and decoders of these L2C solvers typically have the same network architecture. However, a notable difference lies in the number of network layers between the encoder and decoder. For example, Kool et al. [32] adopted the attention layer used in the encoder as the backbone of the decoder, with three attention layers in the encoder but only one in the decoder. This design, commonly known as the head-heavy encoder and light encoder structure, has empirically led to high-level performance on various small-scale problems [18], [73], [95]. However, it is worth noting that Luo et al. [44] recently proposed a novel light encoder (a single layer of attention) and heavy decoder (six attention layers) model (LEHD) to solve large-scale VRPs. LEHD demonstrates reduced sensitivity to the scale of instances and exhibits remarkably improved generalization performance when applied to large-scale VRP instances.

The sequential nature of the AR approach imposes a fundamental speed limit on these L2C solvers [154]. As a result, these solvers struggle to construct high-quality solutions within short time constraints in real-world scenarios [8]. To expedite the decoding process, as shown in Fig. 3, certain L2C solvers [5], [46] employ the non-AR (NAR) approach, enabling the construction of complete solutions in a single decoding step. For example, Xiao et al. [46] employed the Greedy Search algorithm to decode the outputs of the encoder, i.e., the edge score matrix and the start node pointer, adhering to the order constraints inherent in TSP. Although NAR solvers excel in parallel solution constructions with low decoding latency, they often exhibit inferior performance in terms of quality. Subsequently, to obtain NAR solvers with low decoding latency and the capability to generate high-quality solutions, Xiao et al. [5] incorporated a knowledge distillation method to train a NAR solver (student model) using an AR solver (teacher model). For solvers utilizing NAR decoding methods, advocating for more sophisticated postprocessing strategies (see Section III-D for more details), rather than solely relying on the rudimentary Greedy Search algorithm, demonstrates a promising way of enhancing their performance.

In summary, the sequential nature of AR decoding aligns well with the sequential nature of VRPs, where all nodes are visited sequentially, making AR-type decoders predominant. At the same time, the NAR type is still widely adopted to expedite the decoding process.

C. MDP of L2C Solvers

In this section, considering the prevalent adoption of RL among the L2C solvers, we introduce the MDP of L2C solvers. This introduction provides a fresh perspective on understanding the solution construction process of L2C solvers. In addition, the introduction of MDPs of varying solver categories (see Section IV-B for more details about the MDP of L2I solvers) validates the reasonableness and exclusivity of the taxonomy proposed in this article. Specifically, we introduce the MDP of L2C solvers from the following five aspects.

1) State: The state s_t at time t encapsulates three kinds of information, namely the local-level, global-level, and current partial solution information [29]. Specifically, the local-level information is composed of all node-related details, e.g., coordinates and demand. The global-level information encompasses vehicle details, e.g., the remaining capacity of the vehicle for CVRP. The current partial solution information

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:48:06 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

is composed of the previously selected nodes $(a_1, \ldots, a_{t-1})$, where a_{t-1} denotes the node (action) selected at time $t-1$. In addition, when $t = 0$, signifying the initial state, the partial solution is devoid of any selected node.

2) Action: The chosen action a_t at time t denotes a valid node selected from the node set $\boldsymbol{V}$. It is imperative to clarify that nodes adhering to the problem constraints, thus not causing violations, are termed valid nodes. For example, in TSP, the previously visited nodes are considered invalid, and the unvisited nodes are regarded as valid. To ensure the selection of valid nodes, those within set $\boldsymbol{V}$ that could lead to infeasible solutions or violate constraints undergo masking.

3) Transition: The state transition rule is deterministic, involving the addition of the chosen action a_t to the partial solution, thereby transitioning from s_{t-1} to s_t. Subsequently, the state undergoes updates to its local-level, global-level, and partial solution information.

4) Reward: To align the maximization of cumulative rewards with the minimization of the objective value, the reward is typically defined as the negative increment in the objective value (resulting from adding a_t to the current partial solution). Therefore, the value of the cumulative rewards is represented by the negative of the total objective value.

5) Policy: The stochastic policy π selects an action according to the probability of actions acquired through NN learning, given the current state s_t. This process continues until all nodes are selected, concluding the episode and resulting in the construction of a valid tour.

D. Data Augmentation and Postprocessing Strategies

In the field of Computer Vision and Natural Language Processing, data augmentation methods have been shown to be instrumental in significantly elevating model performance. Similarly, data augmentation methods also play a pivotal role in the field of NCO, which can enrich the training datasets and enhance robustness [18], [43], [44], [141]. Specifically, to enrich the training datasets, Kwon et al. [18] were the first to introduce the symmetry coordinate transformation into L2C solvers as a data augmentation method. Subsequently, Kim et al. [90] proposed the orthogonal rotation coordinate transformation method to enrich the training datasets. Following [18], [90], Yao et al. [43] further extended the symmetry of VRPs, and proposed four coordinate transformation methods, namely rotation, symmetry, shrink, and noise. These data augmentation methods enrich the training datasets and remarkably contribute to the high-level performance of L2C solvers. This is particularly evident in the case of SL-based solvers, where a considerable quantity of training instances with high-quality labels is required.

While L2C solvers can construct solutions within seconds using Greedy Search or Beam Search, they are highly prone to falling into the local optima [59]. Specifically, Greedy Search selects the node with the largest probability value at each decoding step, and Beam Search sequentially explores the top k nodes based on their probabilities at each decoding step [31]. Therefore, other than data augmentation methods, postprocessing strategies are also employed to pursue high-quality solutions with a sacrifice of computation time [92], [98], [100], [158]. Initially, in the field of NCO, Bello et al. [92] proposed two kinds of postprocessing strategies, namely sampling and active search (AS). Specifically, Sampling randomly selects nodes utilizing the learned probability during the decoding process, constructs multiple candidate solutions, and ultimately reports the shortest among them. Adopting AS, the model actively updates its parameters for each test instance. Nevertheless, this process of adjusting the weight for each instance is both time-consuming and memory-intensive. To address this problem, Hottung et al. [98] further proposed effective AS (EAS), which selectively adjusts a subset of model parameters. The adoption of EAS remarkably improves the performance of many L2C solvers (e.g., SCA [159]) without substantially increasing time and memory overhead. Subsequently, to couple with EAS while striving for SOTA performance, Choo et al. [157] proposed simulation-guided beam search (SGBS). Specifically, in SGBS, rollouts are performed for the expanded child nodes of Beam Search to identify promising child nodes. Then, the child nodes with poor performance are pruned from consideration, and the beam is expanded to another depth. In future work, techniques conceptually similar to LoRA [160] proposed for large language models (LLMs) could be incorporated to enhance EAS for more efficient solver parameter updates. Recently, Luo et al. [44] proposed the random reconstruct (RRC) postprocessing strategy, which resembles the iterative ruin-and-repair approach used in heuristic algorithms. Specifically, RRC randomly samples a partial solution from the constructed complete solution and restructures it using the proposed NCO solver to obtain a new partial solution.

Finally, it is worth mentioning that certain data augmentation (e.g., symmetry coordinate transformation [18]) and postprocessing strategies (e.g., EAS [98]) also can potentially be integrated into L2I, L2P-O, and L2P-M solvers to pursue better performance [50]. For example, Ma et al. [137] extended the symmetry coordinate transformation technique, integrating it into L2I solvers to improve the diversity of training sets. Owing to the general applicability of data augmentation and postprocessing strategies across all NCO solvers, we refrain from providing repetitive introductions in Sections IV and V.

E. Performance of L2C Solvers

Table III presents the performance of different L2C solvers on small-scale TSP and CVRP instances. Before 2019, most studies focused on enhancing the performance of L2C solvers from the perspective of model architecture. Then, with the establishment of AM [32] and POMO [18] in 2020, the success of these two studies has redirected subsequent research. Instead of exclusively focusing on improving encoders and decoders, there is a distinct shift toward advocating for data augmentation and postprocessing strategies (e.g., SGBS [157]) to enhance the effectiveness of AM and POMO. Through the incorporation of advanced data augmentation and postprocessing strategies, current L2C solvers still showcase performance on par with OR algorithms. For example, SGBS [157] attains gap values of 0.03% and 0.1% on the 100-node TSP and 100-node CVRP, respectively. However, existing L2C solvers have certain inadequacies when faced with specific situations. First, it is worth mentioning that certain L2C solvers prioritized high performance over solving time, leading to the conventional OR algorithms achieving shorter solution time [46]. This contradicts the inherent advantage of NCO solvers, i.e., their ability to solve VRP instances rapidly. In addition, the generalization capability of these L2C solvers needs to be enhanced to better solve instances with different data distribution patterns and problem scales (see Section VI-A for more details). Finally, due to the time and memory complexity,

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:48:06 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE III

PERFORMANCE OF VARIOUS L2C SOLVES ON SMALL-SCALE TSP AND CVRP INSTANCES

Methods	TSP ($N = 50$)			TSP ($N = 100$)			CVRP ($N = 50$)			CVRP ($N = 100$)		
	Obj.	Gap	Time	Obj.	Gap	Time	Obj.	Gap	Time	Obj.	Gap	Time
Concorde (2007) [155]†	5.696	-	9m	7.765	-	43m	-	-	-	-	-	-
HGS (2022) [156]†	-	-	-	-	-	-	10.366	-	1.2d	15.563	-	1.5d
Ptr-Net-SL (2015) [31] (greedy)	7.660	34.48%	-				-	-	-	-	-	-
Ptr-Net-RL (2017) [92] ($S = 1280$)	5.750	0.95%	-	8.000	3.03%	-	-	-	-	-	-	-
S2V-DQN (2018) [93] (greedy)	-	-	-	8.310	7.03%	-	-	-	-	-	-	-
AM (2019) [32] ($S = 1280$)	5.730	0.52%	-	7.940	2.26%	-	10.620	-	-	16.230	-	-
POMO (2020) [18] ($A = 8, S = 200$)†	5.696	0.00%	1.1h	7.770	0.07%	5.6h	10.397	0.30%	1.4h	15.672	0.70%	7.2h
HAM (2021) [106] ($B = 10$)	-	-	-	-	-	-	10.490	-	-	17.110	-	-
MDAM (2021) [59] ($B = 250$)	5.700	0.03%	-	7.790	0.38%	-	10.480	-	-	15.990	-	-
AM-ASW (2021) [95] (greedy)	5.760	1.16%	-	8.010	3.20%	-	10.900	-	-	16.420	-	-
POMO-EAS (2022) [98] ($A = 8, S = 200$)†	5.696	0.00%	2h	7.769	0.05%	10.9h	10.379	0.13%	3.1h	15.610	0.30%	16h
POMO-Sym (2022) [90] (A=8, S=200)†	-	-	-	7.771	0.08%	5.6h	-	-	-	15.702	0.89%	7.2h
POMO-SGBS (2022) [157] ($A = 8, S = 200$) †	-	-	-	7.767	0.03%	1.1d	-	-	-	15.579	0.10%	4.1d
Pointerformer (2023) [141] ($A = 8, S = 200$)†	5.697	0.02%	1.1h	7.773	0.11%	5.6h	-	-	-	-	-	-
SL-DABL (2023) [43] ($A = 8, S = 200$)	5.693	0.00%	-	7.769	0.05%	-	-	-	-	-	-	-
FER-POMO (2023) [153] ($A = 8, S = 200$)	-	0.00%	-	-	0.03%	-	-	0.16%	-	-	0.56%	-
LEHD (2023) [44] ($RRC = 100$)	-	-	-	-	0.01%	-	-	-	-	-	0.27%	-

Note: Symbol † denotes the results are taken from [50], while the rest are taken from the given references. Abbreviations A, S, and B denote the number of augmentations, samplings, and width of the Beam Search, respectively. In addition, greedy signifies the adoption of Greedy Search during the decoding process. For all experiments on TSP and CVRP, "Obj.", "Gap", and "Time" denote the average objective values, average gaps, and the total run time on the corresponding test dataset with 10k instances, respectively. The gaps are computed relative to the exact algorithm Concorde [155] for TSP and the SOTA OR algorithm HGS [156] for CVRP.

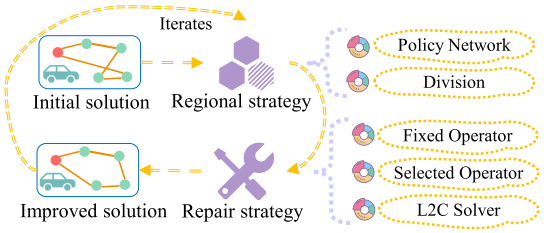


Fig. 5. Illustration of the iterative improvement solutions process of L2I solvers, starting from an initial complete solution and ending within a given timeframe. L2I solvers first rely on regional strategies to select regions (typically node pairs). Subsequently, diverse strategies are employed to repair the subtours of the selected regions.

existing transformer-based L2C solvers struggle with handling large-scale VRP instances, e.g., the 10 000-node TSP (see Section VI-B for more details).

IV. L2I SOLVERS

Inspired by the idea of ruining and repairing in heuristic algorithms, L2I solvers iteratively improve current complete solutions, facilitating the exploration of (sub)optimal solutions within a specified time frame, as shown in Fig. 5. Specifically, at each time step, the regional strategy of L2I solvers identifies the regions (typically node pairs) requiring "repair." Subsequently, L2I solvers exploit the repair strategy in the selected regions to improve current complete solutions. To thoroughly review L2I solvers, Sections IV-A and IV-B outline the designs of regional strategy and three repair strategies, respectively. Subsequently, Sections IV-C and IV-D introduce the MDP of L2I and provide a comprehensive overview of the recent L2I solvers' performance on small-scale TSP and CVRP instances, respectively.

A. Design of Regional Strategies

L2I solvers generally use a policy network to identify the regions that require repair (in other words, performance elevation) based on current solutions [6]. In the field of NCO, Chen and Tian [47] first adopted a fully connected NN as a score predictor to identify a few nodes where rewriting might be beneficial. Subsequently, Costa et al. [49] and Sui et al. [161] adopted a network block consisting of GNN and RNN to select nodes based on current solutions. Considering the superior performance of transformer-based networks for VRP, Wu et al. [48] proposed a transformer-based network to choose node pairs for executing the 2-opt operator (see Section IV-B for more details about the 2-opt operator). Subsequently, Ma et al. [89] proposed DACT to employ two transformer networks to individually learn information from node coordinates and current solutions. Such information is then collaboratively used to determine the selection of node pairs. However, due to the adoption of two transformer-based networks, the computing efficiency of DACT is not satisfactory [29]. To improve computing efficiency, Ma et al. [137] further proposed the synthesis attention mechanism, capable of simultaneously learning embeddings of node coordinates and current solutions to attain a comprehensive representation. While this regional strategy typically yields satisfactory results for small-scale VRPs, its efficiency diminishes when used to solve large-scale VRPs. This is because using NNs to select regions consisting of only a few nodes in large-scale VRPs could exponentially increase the overall solution time, ultimately leading to inefficient solution provision.

To better tackle large-scale VRP instances, certain L2I solvers [162] propose a division strategy inspired by the D&C-based manner, rather than relying on NNs to select regions. Specifically, these D&C-based L2I solvers utilize the visiting order of the current complete solution to divide the problem into multiple subproblems. Subsequently, all these subproblems are simultaneously entered into the repair strategy for performance elevation. Finally, only the subproblems showing improvement are considered for iteratively updating the current solutions. Moreover, to simultaneously improve multiple subproblems, these L2I solvers [70], [162] also design a special repair strategy (see Section IV-B for more details). For these solvers, to prevent increased distance costs during the merging of subproblems, establishing an efficient division strategy is crucial. Future research may explore the use of visual object detection models (e.g., YOLO [163]) to facilitate region division, as these models are capable of capturing visual

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:48:06 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

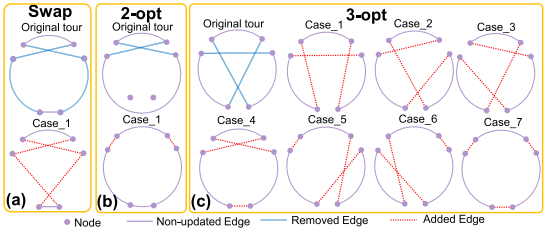


Fig. 6. Illustration of the exchange processes for (a) swap, (b) 2-opt, and (c) 3-opt operators. The blue and red links denote the edges selected to be removed and added to the current tour, respectively. For swap and 2-opt operators, only one exchange takes place. However, for a 3-opt operator, seven different exchanges take place.

patterns, e.g., spatial structures and local node concentrations, which may lead to more effective partitions in the solution space and thereby reduce the cost of merging subproblems.

In summary, numerous L2I solvers favor transformer-based NNs to identify regions necessitating repair in small-scale VRP instances. In addition, to handle large-scale VRP instances, certain L2I solvers divide the problem into multiple subproblems and strive to improve all subproblems.

B. Design of Repair Strategies

Currently, L2I solvers employ one of three distinct repair strategies to repair subtours of the selected regions. Initially, for simplicity, certain L2I solvers exploit the fixed operator throughout the entire iteration process to improve current solutions. For example, Chen and Tian [47] adopted the swap operator to iteratively improve current solutions, involving the exchange of the visiting order of two nodes, as shown in Fig. 6(a). Subsequently, Wu et al. [48] and Costa et al. [49] adopted the 2-opt operator for iterative improvements in current solutions. As shown in Fig. 6(b), the 2-opt operator reconnects two new edges at a time. Despite the simplicity of swap and 2-opt operators, their performance falls short in searching for (sub)optimal solutions within a limited timeframe (see Section IV-D).

To pursue better performance, certain L2I solvers exploit additional NNs to choose suitable operators for the regions selected by the regional strategy. For example, Lu et al. [105] exploited a transformer-based network to select operators from two superior sets customized for CVRP, namely intra-route and inter-route sets. Specifically, intra-route operators endeavor to alter the order of visiting nodes within a single route, while inter-route operators attempt to exchange nodes among different routes. Meanwhile, Sui et al. [161] replaced 2-opt with 3-opt and proposed an NN to choose from seven different cases of the 3-opt operator, as depicted in Fig. 6(c). Generally speaking, the operator's performance improves with a larger value of k in k-opt operators, but simultaneously, the computing complexity significantly increases. To automatically determine the values of k at different iteration stages, Ma et al. [50] proposed NeuOpt, which decomposes the sequential k-opt operator into a structured sequence consisting of a starting move, k intermediate moves (determined by the proposed repair network), and an ending move. This design effectively balances coarse-grained (larger k) and fine-grained (smaller k) search strategies.

Finally, to tackle large-scale VRP instances, certain L2I solvers (e.g., SO [70] and GLOP [162]) exploit L2C solvers (e.g., AM [32]) to directly repair all subproblems, bypassing operators like the 2-opt operator. The reason for employing L2C solvers as the repair strategy lies in the regional strategy utilized by these L2I solvers. These solvers divide large-scale problems into multiple smaller subproblems, which can be efficiently solved by L2C solvers in parallel. In contrast, traditional operators lack the capability to simultaneously solve a batch of subproblems. In addition, Li et al. [164] proposed DRHG, which first decomposes the complete solution into certain isolated nodes and segments (with each segment composed of multiple consecutive nodes). Subsequently, DRHG simplifies the segments into hyperedges (temporally retaining only the two endpoints) and utilizes the L2C solver to connect the endpoints of these hyperedges with the isolated points, ultimately reconstructing a complete solution. This design directs the L2C solver's attention to the disrupted regions while reducing the encoding complexity of all nodes within the segments, particularly suited to handle large-scale VRP instances. In addition, it is worth mentioning that while these solvers employ L2C solvers as their repair strategy, their key objective remains to iteratively improve the current complete solutions. Hence, we categorize these solvers into L2I categories.

In summary, to pursue better performance, many recent L2I solvers employ NNs to select suitable operators instead of relying on a fixed single one. Because of the requirement for improving multiple subproblems in parallel, certain L2I solvers incorporate L2C solvers to repair them.

C. MDP of L2I Solvers

In this section, to elucidate the fundamental distinction between L2C solvers and L2I solvers, we introduce the MDP of L2I solvers from the following aspects.

1) State: The state s_t reflects three kinds of information, namely local-level, global-level, and current solution information at time t. Local-level information encompasses details about all nodes (e.g., coordinates) [29]. The global-level information consists of historical solutions and their corresponding costs. Unlike the MDP of L2C solvers, the current solution information δ_{t-1} of the MDP for L2I solvers is a complete solution. In addition, when $t = 0$, the initial solution δ_0 can be any randomly generated feasible solution.

2) Action: The action a_t represents a predefined operation that transforms the current solution into an alternative one. For example, one form of action involves specifying nodes to perform a pairwise local search operation (e.g., 2-opt operator) in [48] and [89]. Analogous to L2C solvers, ensuring a consistently feasible solution involves applying a mask mechanism to the action space in L2I solvers.

3) Transition: In numerous L2I solvers, the state transition involves updating the current solution δ_{t-1} with the improved solution δ_t. Simultaneously, the information at both the local and global levels of the state undergoes corresponding updates.

4) Reward: To ensure that cumulative rewards accurately capture the overall improvement, i.e., a lower objective value from the initial solution, the reward is commonly defined as the immediate reduction in the objective value of the current best solution following action a_t.

5) Policy: The policy π of the MDP for L2I solvers is parameterized by an NN in most cases. Unlike L2C solvers, the number of iterations of the L2I solvers T could be any predefined value. Therefore, if T varies, comparing different L2I solvers becomes inequitable. Finally, the best solution

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:48:06 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE IV
PERFORMANCE OF VARIOUS L2I SOLVES ON SMALL-SCALE TSP AND CVRP INSTANCES

Methods	TSP ($N = 50$)			TSP ($N = 100$)			CVRP ($N = 50$)			CVRP ($N = 100$)		
	Obj.	Gap	Time	Obj.	Gap	Time	Obj.	Gap	Time	Obj.	Gap	Time
Concorde (2007) [155]†	5.696	-	9m	7.765	-	43m	-	-	-	-	-	-
HGS (2022) [156]†	-	-	-	-	-	-	10.366	-	1.2d	15.563	-	1.5d
NeuRewriter (2019) [47] ($T = 40K$)	-	-	-	-	-	-	10.510	-	-	16.100	-	-
N2I (2020) [105] ($T = 40K$)	-	-	-	-	-	-	10.350	-	-	15.570	-	-
NLNS (2020) [104] ($T = 5K$) †	-	-	-	-	-	-	10.506	1.35%	1.4h	15.915	2.26%	2.4h
N2OPT (2020) [49] ($T = 2K$) †	5.703	0.12%	40m	7.824	0.77%	1.1h	-	-	-	-	-	-
N3OPT (2021) [161] ($T = 2K$) †	5.700	0.08%	48m	7.820	0.74%	1.3h	-	-	-	-	-	-
MT (2021) [48] ($T = 5K$) †	5.709	0.23%	1.3h	7.884	1.54%	2h	10.544	1.72%	4.2h	16.165	3.87%	5h
DACT (2021) [89] ($T = 10K$) †	5.696	0.00%	2.7h	7.772	0.10%	7h	10.383	0.16%	16h	15.736	1.11%	1.6d
NeuOpt (2023) [50] ($T = 10K$) †	5.696	0.00%	1.1h	7.766	0.02%	2.8h	10.375	0.08%	2h	15.656	0.60%	4.6h
DRHG (2025) [164] ($T = 1K$)	-	-	-		0.00%	-	-	-	-	-	-0.02%	-

Note: Symbol † denotes the results are taken from [50], while the rest are taken from the given references. Abbreviation T denotes the number of iterations.

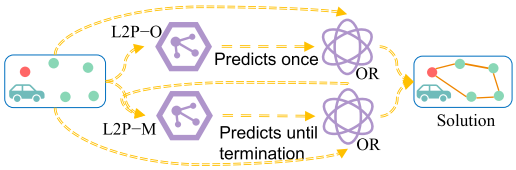


Fig. 7. Illustration of the information prediction processes of L2P-O and L2P-M solvers. L2P-O solvers aim to enhance OR algorithms by providing valuable information solely before the searches begin. Subsequently, OR algorithms solve VRP instances by leveraging the predicted information along with problem definitions. Conversely, L2P-M solvers collaborate with OR algorithms continuously during the search processes, offering prediction of key information at each decision step. These L2P-M solvers take states of the OR algorithms, encompassing problem definitions, as their inputs [55].

identified across the entire iteration process is reported as the final solution.

D. Performance of L2I Solvers

Table IV presents the performance of various L2I solvers on small-scale TSP and CVRP instances. Notably, NeuOpt [50], the most novel L2I solver utilizing the k-opt operator, outperforms the others, demonstrating only 0.02% and 0.6% gaps compared to Concorde and HGS, respectively. However, for L2I solvers, it is crucial to acknowledge that the problem of unfair comparisons becomes more pronounced when employing a varying number of iterations T. For a fair comparison, adopting a fixed number of iterations for all L2I solvers is imperative [22]. In addition, the solution time also undergoes a significant rise with an increase of T. Finally, existing L2I solvers typically begin with randomly initiated feasible solutions and iteratively improve them for straightforwardness. However, this random initialization strategy hinders convergence and increases search overhead, particularly for large-scale VRP instances [165]. Future studies should explore more efficient strategies borrowed from the heuristic algorithms for constructing initial solutions to improve the time efficiency of L2I solvers.

V. L2P SOLVERS

In many cases, relying solely on NNs to address VRPs may not be the optimal strategy. Instead, there is an emerging advocacy for integrating NNs with OR algorithms, as depicted in Fig. 7. This integration aims to expedite the search process of OR algorithms and enhance the quality of solutions by leveraging DL methods to predict crucial information [21], [56], [166]. Sections V-A and V-B comprehensively review L2P-O and L2P-M solvers, respectively. Subsequently, Section V-C presents the performance of existing L2P-O and L2P-M solvers on small-scale VRP and CVRP instances.

A. L2P-O Solvers

A group of L2P-O solvers predicts information only once before commencing searches with OR algorithms, aiming to expedite the search processes. For example, to confine the search space and thus expedite the search processes of DP, Kool et al. [52] adopted a graph convolution network (GCN) to predict a sparse heatmap for each TSP instance, which highlights promising edges. Subsequently, DP excludes solutions with fewer "heated" edges, effectively eliminating a multitude of inappropriate solutions and confining the search space, thereby reducing runtime. Similar to [52], other studies [167], [168], [169] also employed L2P-O solvers to predict the scores of edges, effectively confining the search space and accelerating the search processes. In addition, it is worth mentioning that certain studies [45], [79], [170], [171] adopted search algorithms, e.g., Monte Carlo Tree Search (MCTS) and Beam Search, rather than OR algorithms, to construct solutions based on the predicted heatmaps. Nevertheless, because the solution construction process of these methods employing search algorithms (e.g., [45]) is closely similar to OR algorithms (e.g., [52]), we categorize these methods as L2P-O solvers as well. For example, Joshi et al. [45] leveraged GCN to predict heatmaps and employed Beam Search on the resulting heatmaps to construct solutions. Specifically, they formulated TSP instances as 2-D Euclidean graphs and utilized the optimal solutions as labels for training GCN to predict heatmaps. While constructing solutions using search algorithms is more efficient compared to the OR algorithms, it comes with the inadequacy of being unable to solve multi-constrained VRPs.

Meanwhile, other L2P-O solvers predict information to derive higher quality solutions. For instance, Xin et al. [51] trained a GNN for predicting edge scores and node penalties, resulting in a more potent edge candidate set than the one created by the original minimum spanning tree used in LKH [63]. More recently, Ye et al. [172] proposed deep ant colony optimization (DeepACO), which substitutes human-designed heuristic measures (e.g., Euclidean distances of nodes) in ACO with superior heuristic measures predicted by GNN. Finally, certain methods exploit NNs to configure parameters for OR algorithms or select the optimal OR algorithm for each instance, significantly improving the quality of the derived solutions [173], [174], [175], [176]. For example, Gutierrez-Rodríguez et al. [175] defined a set of meta-features

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:48:06 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

that appropriately characterize different VRP instances. Subsequently, they employed a fully connected NN to predict the most suitable OR algorithm for each instance.

In summary, certain L2P-O solvers exploit the predicted information to expedite the search processes of OR algorithms, while others leverage such information to enhance the quality of solutions generated by OR algorithms. Meanwhile, in addition to the advantages aforementioned, it should not be overlooked that L2P-O solvers also face the generalization capability inadequacy, similar to L2C and L2I solvers.

B. L2P-M Solvers

Compared with L2P-O solvers, L2P-M ones exploit NNs to make the same type of decisions multiple times, aiming to enhance the performance of conventional OR algorithms in terms of time complexity. For example, to expedite search processes, Zheng et al. [55] substituted the inflexible process of traversing the candidate set to select edges in the k-opt operator of LKH [63]. Instead, an RL-based method is employed to predict key information and guide the decision-making process for edge selections within the candidate set. Meanwhile, certain L2P-M solvers accelerate the search processes of DP [177], [178]. For example, Xu et al. [178] introduced a general framework called NDP, which exploits fully connected NNs to replace DP's value or policy functions at each decision step. It is worth mentioning that although having the same purpose as [52], the L2P-M solvers [177], [178] exploited the NN to predict multiple pieces of information for each TSP instance in the whole process of DP. In contrast, the L2P-O solver DPDP [52] produces heatmaps only once for each TSP instance. Hence, the proposed taxonomy is still applicable in this case to exclusively divide these NCO solvers into respective types.

To strive for high-quality solutions, Hottung et al. [54] exploited an NN to predict the latent search space with a smooth landscape, then employed OR algorithms (e.g., differential evolution (DE) [186]) to search in the predicted space. Recently, Liu et al. [187] proposed a framework based on LLMs to automatically generate multiple heuristic algorithms for producing high-quality solutions for VRPs. Similar to evolutionary computation (EC), the proposed LLM framework encompasses initialization, selection, crossover, and mutation steps. However, unlike EC, where each individual represents a solution of a VRP instance, in the LLM framework, each individual embodies a heuristic algorithm [187]. In addition, during each iteration, crossover and mutation operators involve employing prompts (comprising task descriptions, parent individuals, and generation instructions) to guide LLMs in generating multiple heuristic algorithms. Following [187], Ye et al. [182] further proposed two innovative prompt methods for crossover and mutation operators, respectively. The short-term prompt is employed for the crossover operator, compelling LLMs to analyze the differences in performance between the two parents as search directions. Conversely, the long-term prompt is utilized for the mutation operator, prompting LLMs to assimilate insights from historical individuals. To enable LLMs to provide specific search directions, Wu et al. [183] proposed the core abstraction prompting method, which abstracts the core components from elite heuristics and incorporates them as prior knowledge in short- and long-term prompts. Future research could further encourage LLMs to develop more sophisticated heuristic algorithms that may yield superior performance. In particular, we believe that further improvement is achievable by fine-tuning LLM to specialize in generating heuristics. Moreover, a prior study [188] suggested that heuristics can enhance the performance of NCO solvers. This also represents a promising direction for improving the generalization ability of NCO solvers across different distributions and scales by deriving tailored heuristics through LLMs.

In conclusion, akin to L2P-O solvers, L2P-M solvers are extensively employed to expedite OR algorithm search processes or improve solution quality. However, generally speaking, L2P-M solvers tend to be more time-consuming in similar situations due to their multiple-prediction nature.

C. Performance of L2P-O and L2P-M Solvers

Table V presents the performance of existing L2P-O and L2P-M solvers on small-scale TSP and CVRP. The results demonstrate that by leveraging the advantages of the conventional OR or search algorithms, most L2P-O and L2P-M solvers exhibit commendable performance on small-scale VRP instances, such as the 50-node TSP. Furthermore, these L2P-O and L2P-M solvers exhibit proficiency in handling large-scale TSP instances, outperforming L2C and L2I solvers (see Section VI-B for more details). However, other than DPDP [52] and DeepACO [172], the remaining heatmap-based L2P-O solvers, including GCN [45], UTSP [79], Att-GCN [179], DIMES [180], DIFUSCO [53], and T2TCO [181], encounter difficulties when addressing CVRP. This difficulty arises from the additional constraints imposed in CVRP, i.e., the varying goods demand for each node and a vehicle capacity limit. The intricate constraints of VRP variants cannot be adequately modeled solely through the predicted heatmaps and search algorithms [50]. In future research, the exploration of integrating more robust OR algorithms is warranted to address the diverse constraints inherent in various VRP variants and improve the overall performance of heatmap-based solvers.

VI. Inadequacies, Efforts, and Perspectives of Existing NCO Solvers

Despite the high-level performance achieved by existing NCO solvers in terms of the quality of solutions and time complexity, they remain deficient when compared to the conventional OR algorithms [22], [189], [190], [191], [192]. Through a systematic survey of current research hotspots (see Section I), we identify the open inadequacies of SOTA NCO solvers that primarily stem from distinctive VRP requirements in practical applications and introduce them in Sections VI-A–VI-D, respectively. This section also discusses existing efforts to solve these inadequacies, identifies open inadequacies, and proposes additional promising directions to address them.

A. Inadequacies of Generalization Capability

The prevailing inadequacy among the majority of existing NCO solvers is their limited capacity for generalization across diverse data distribution patterns [193], [194], [195], [196], [197], [198]. For example, solvers trained with instances following the uniform distribution [i.e., $\forall v_i \in \boldsymbol{V}, v_i \sim \mathcal{U}([0,1]^2)$] are less likely capable of handling instances conforming to the normal distribution [i.e., $\forall v_i \in \boldsymbol{V}, v_i \sim \mathcal{N}([0,1]^2)$] [67]. This inadequacy impedes the application of NCO solvers in real-world problems [29].

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:48:06 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE V
PERFORMANCE OF VARIOUS L2P SOLVES ON SMALL-SCALE TSP AND CVRP INSTANCES

Methods	Type	TSP ($N = 50$)			TSP ($N = 100$)			CVRP ($N = 50$)			CVRP ($N = 100$)		
		Obj.	Gap	Time	Obj.	Gap	Time	Obj.	Gap	Time	Obj.	Gap	Time
Concorde (2007) [155]†	OR	5.696	-	9m	7.765	-	43m	-	-	-	-	-	-
HGS (2022) [156]†	OR	-	-	-	-	-	-	10.366	-	1.2d	15.563	-	1.5d
GCN (2019) [45]†	L2P-O/BS	5.698	0.04%	23m	7.869	1.35%	46m	-	-	-	-	-	-
Att-GCN (2021) [179]†	L2P-O/MCTS	5.691	0.01%	8m	7.764	0.04%	15m	-	-	-	-	-	-
CAVE-Opt-DE (2021) [54]†	L2P-M/DE	-	0.02%	2.5d	-	0.34%	1.8d	10.400	-	4.7d	15.750	-	11d
VSR-LKH (2021) [55]	L2P-M/LKH	-	-	-	7.765	0.01%	-	-	-	-	-	-	-
NeuroLKH (2021) [51]	L2P-O/LKH	-	-	-	7.765			-	-	-	15.575	-	-
GNN (2022) [170] †	L2P-O/GLS	-	0.00%	2.8h	-	0.58%	2.8h	-	-	-	-	-	-
DPDP (2022) [52] †	L2P-O/DP	-	-	-	7.765	0.00%	1.9h	-	-	-	15.627	0.41%	1.2d
DIMES (2022) [180] †	L2P-O/MCTS	-	-	-	7.765	0.01%	-	-	-	-	-	-	-
UTSP (2023) [79]	L2P-O/BFLS	5.696	0.00%	-	7.765	0.00%	-	-	-	-	-	-	-
DIFUSCO (2023) [53] †	L2P-O/2-opt	5.696	0.01%	5.8h	7.766	0.02%	21.7h	-	-	-	-	-	-
T2TCO (2023) [181]	L2P-O/2-opt	5.696	0.02%	-	7.767	0.06%	-	-	-	-	-	-	-
DeepACO (2023) [172]	L2P-O/ACO	-	-	-	7.767	-	-	-	-	-	15.770	-	-
EOH (2024) [57]	L2P-M/GLS	-	0.00%	-	-	0.03%	-	-	-	-	-	-	-
ReEvo (2024) [182]	L2P-M/GLS	-	0.00%	-	-	0.00%	-	-	-	-	-	-	-
Hercules (2025) [183]	L2P-M/GLS	-	0.00%	-	-	0.00%	-	-	-	-	-	-	-

Note: Symbol † denotes the results are taken from [50], while the rest are taken from the given references. Abbreviations GLS and BFLS denote Guided Local Search [184] and Best-first Local Search [185], respectively.

To address the inadequacy of impaired generalization ability, certain L2C solvers leveraged more diverse instances as training sets, compelling solvers to learn from varying types of distribution [66], [68], [199], [200], [201]. For example, Jiang et al. [68] collectively optimized the parameters of solvers using instances from different distribution patterns. Meanwhile, Zhang et al. [199] incorporated curriculum learning into L2C solvers, which utilize instances with diverse distribution patterns to progressively train solvers. In contrast to the aforementioned approaches that focused on the diversity of instance distribution patterns, Bi et al. [67] tackled the inadequacy of generalization at the model level. Specifically, they incorporated the knowledge distillation method into NCO, utilizing multiple teacher models trained on different types of distributions to guide a generalized student model capable of performing effectively across varying distribution patterns.

In addition, NCO solvers exhibit limited generalization capability across different problem scales simultaneously [66]. For example, those trained on trivially small scales struggle to generalize to larger instances. While the approaches introduced in the previous paragraph solely focus on distribution, certain studies [202], [203] simultaneously addressed generalization concerning both the problem scale and distribution. Specifically, Manchanda et al. [202] formalized solving VRP instances over a given instance distribution and scale as a separate learning task. Subsequently, they utilized meta-learning to learn a model capable of adapting to various tasks. Following [202], Zhou et al. [203] replaced the random task sampling method used in [202] with a hierarchical scheduler to adaptively select different types of tasks. In addition, to mitigate the computing overhead, they introduced a second-order derivative approximation method within the meta-learning framework. This approach entails utilizing second-order derivatives primarily in the early stages of training, and subsequently relying exclusively on their first-order counterparts.

While considerable advancement has been made in enhancing the generalization of NCO solvers, it is crucial to acknowledge that these generalization methods require more hyperparameters and efforts on data synthesis, resulting in a significant overhead in computing resources. We propose employing more lightweight NNs to improve generalization capability. For example, Xiao et al. [204] proposed a plug-and-play entropy-based scaling factor to adjust attention scores, thereby enhancing the generalization capability of the solver while requiring fewer computing resources. In addition, recent studies [69], [102], [103], [192] emphasized the role of nearest-neighbor relationships among nodes in capturing local invariance across problems of varying scales and distributions. This approach is particularly effective because, regardless of the instance distribution or scale, nodes in (pseudo-)optimal solutions are highly likely to connect to their nearest neighbors [205], [206], [207]. Such consistent local connectivity patterns provide a stable and transferable prior, thereby enhancing the model's generalization ability. Moreover, future research may be dedicated to improving generalization by enriching node features with additional information (e.g., the distance from the candidate node to the current node or the average distance from the candidate node to all unvisited nodes), rather than solely relying on node coordinates.

Currently, certain solvers achieve high-level performance on synthetic TSP datasets across different distributions and scales. For instance, the SOTA solver GELD [65] attains gap values of 2.38%, 3.44%, 4.7%, and 2.81% on 10 000-node TSP instances with uniform, clustered, explosion, and implosion distribution patterns, respectively. However, on real-world instances of the same scale, such as those in TSPLib[2] and National TSPs,[3] GELD's gap value remains high at 7%, highlighting the need for further improvement of NCO solvers on real-world instances. This inferior performance stems from the fact that these existing solvers fall under the paradigm of domain adaptation and are trained on various known distribution patterns to enhance generalization. However, such strategies may not transfer well to real-world instances because real-world distribution patterns are typically unknown or difficult to model. Future research should explore domain generalization strategies (e.g., invariant learning) that focus on features invariant to environmental variations, thereby improving out-of-distribution performance on real-world instances. In addition, generating synthetic instances with more diverse and realistic structural characteristics constitutes another promising research direction for bridging the gap between synthetic and real-world scenarios. Finally, drawing from the emphasis on enhancing the generalization capacity of L2C solvers in prior studies, attention to other solver categories, such as L2I, L2P-O, and L2P-M solvers, has been relatively scarce. We posit that with the help of OR algorithms, L2P-O and L2P-M

[2]URL: http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/
[3]URL: https://www.math.uwaterloo.ca/tsp/world/countries.html

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:48:06 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

solvers could potentially demonstrate superior generalization capabilities.

B. Inadequacies of Solving Large-Scale VRPs

In addition to the inadequacy posed by poor generalization across different problem distributions and scales, many existing solvers encounter difficulties when handling large-scale VRPs in real-time [208], which limits their applicability to real-world instances with large numbers of nodes. This is particularly notable for L2C and L2I solvers [99], [209], despite their demonstrated ability to outperform or be comparable to the conventional OR algorithms with up to 100 nodes [18]. The underlying reason lies in the quadratic growth of time complexity and memory space associated with NNs in most solvers as the problem scale increases [44]. One prominent example is the self-attention mechanism, a widely adopted NN in NCO solvers. Self-attention mechanism is defined based on query Q, key K, value V, where $Q, K, V \in \mathbb{R}^{N\times d}$, with N and d denoting the problem scale and predefined dimension, respectively. The embedding h_i for the ith node is computed using the following Softmax function:

$$h_i = \sum_j \frac{\exp\left(q_i k_j^\top / \sqrt{d}\right)}{\sum_l \exp\left(q_i k_l^\top / \sqrt{d}\right)} v_j \tag{15}$$

where $\exp(\cdot)$ and $\top$ denote exponential and transpose functions, respectively. Vectors q_i, k_i, and v_i denote the query, key, and value of the ith node, respectively. According to (15), it is evident that each instance involves a dot product computation with a complexity of $\mathcal{O}(N^2 d)$ and a memory usage complexity of $\mathcal{O}(N^2)$ [146]. When the problem scale N increases to λN, where λ denotes an integer greater than one, the complexity of dot product computation and memory usage grow to $\mathcal{O}(\lambda^2 N^2 d)$ and $\mathcal{O}(\lambda^2 N^2)$, respectively. Hence, as the problem scale expands, solvers employing the self-attention mechanism are prone to memory depletion during training, hampering convergence [44]. Similarly, various other NNs experience problem scale-related effects to varying degrees. Moreover, when VRPs include multiple constraints (e.g., heterogeneous CVRP, where each vehicle has different capacity and speed attributes [210]), encoding those constraints typically requires more complex network architectures, thus substantially increasing memory consumption. Therefore, existing solvers face greater challenges in handling large-scale multiconstraint VRPs than large-scale VRPs with few or no constraints.

To address this issue, certain methods have embraced the D&C manner to decompose large-scale problems into multiple small-scale subproblems [71], [91], [162], [179], [211]. Subsequently, these methods employ diverse solvers to construct or improve solutions for all decomposed subproblems, followed by the integration of all subproblems. For example, Pan et al. [71] proposed H-TSP, which utilizes an upper level partition model and a lower level L2C solver for partitioning and solving subproblems, respectively. Similarly, Hou et al. [91] exploited a transformer to divide a problem into multiple subproblems and incorporated LKH3 to construct solutions for each subproblem. For these D&C solvers, identifying the optimal decomposition strategy for large-scale VRPs stands as a crucial factor [77], [162].

While pursuing the same objective, other NCO solvers [79], [141], [149], [213] diverge from the D&C manner in addressing the time and memory expenses associated with solving large-scale VRPs. Specifically, they adopt lighter NNs or innovative training paradigms. For example, Yang et al. [149] replaced scaled dot-product attention with sampled scaled dot-product attention (see Section III-A for more details). Xiao et al. [65] proposed the region average linear attention mechanism, which partitions nodes into regions to facilitate efficient global information exchange, achieving a computation complexity $\mathcal{O}(Nd)$. In addition, to optimize memory usage, Luo et al. [44] adopted SL as the training paradigm to train an NN rather than RL. This choice stems from the fact that RL training necessitates generating the complete solution before computing the reward, implying a substantial demand for memory and computing resources. More recently, certain studies [214], [215], [216] exploited the high-quality solutions produced by the solver as labels, enabling self-SL. While this approach partially alleviates the challenges of label scarcity in SL and slow convergence in RL, it still suffers from low sample utilization. During training, all sampled solutions except for the optimal one are discarded, thereby failing to effectively exploit the potential informational value of suboptimal solutions. Efficiently leveraging a large number of suboptimal solutions remains a critical and open challenge in this line of work. Furthermore, in contrast to the original study [45], which involves training a GCN using SL, Min et al. [79] introduced an innovative UL loss function to train GCN. By adopting this design, GCN utilizes approximately 10% of the number of parameters compared to the SL training paradigm.

Table VI presents the performance of NCO solvers designed for solving large-scale TSP and CVRP instances. For TSP, comparing L2C solvers that abstain from employing the D&C approach (TSPformer [149] and Pointerformer [141]) with their counterparts utilizing this strategy (H-TSP [71]) exposes a consistent performance gap ranging from 1% to 10%. However, even though H-TSP adopts the D&C manner, its capability is limited to handling TSP instances with a maximum of 1000 nodes. In contrast, L2P-O solvers outperform L2C solvers, especially the recently proposed heatmap-based solvers (e.g., DIFUSCO [53], T2TCO [181], and Fast T2T [217]) that can achieve high-level performance in real-time, with gaps falling within the 1% range, when dealing with 1000-node TSP instances. Notably, on the challenging 10 000-node TSP, DIFUSCO [53] achieved an impressive gap value of 2.58%, credited to the training method exploiting the diffusion model. Specifically, DIFUSCO constantly adds Bernoulli noises to optimal solutions of instances and compels a GCN to denoise the added noises. Through this training method, GCN has the capability to predict high-quality heatmaps. Subsequently, MCTS is employed to construct (sub)optimal solutions based on these high-quality heatmaps. To accelerate inference, Wang et al. [212] proposed DEITSP, which optimizes multiple generation steps of DIFUSCO into a single step, transforming Bernoulli noise into high-quality heatmaps through a specially designed bimodal graph transformer for one-step prediction. In addition, the stochastic noise-injection and denoising process in diffusion model-based solvers can produce irregular and nonsmooth transitions, resulting in inferior performance. Future research may consider adopting the flow matching paradigm [218] to explicitly model discrete solutions, thereby reducing reliance on stochastic denoising and improving solution quality and stability.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:48:06 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE VI
PERFORMANCE OF VARIOUS NCO SOLVES ON LARGE-SCALE TSP AND CVRP INSTANCES

Methods	Type	TSP ($N = 500$)			TSP ($N = 1,000$)			TSP ($N = 10,000$)			CVRP ($N = 1,000$)		CVRP ($N = 2,000$)	
		Obj.	Gap	Time	Obj.	Gap	Time	Obj.	Gap	Time	Obj.	Time	Obj.	Time
LKH3 (2021) [13] ‡	OR	16.55	-	5.5m	23.12	-	24m	71.77	-	13h	46.40	6.2s	64.90	20s
Att-GCN (2021) [179] ‡	L2P-O/D&C/MCTS	16.96	2.48%	2.9m	23.86	3.20%	6.1m	75.73	5.50%	13.1m	-	-	-	-
L2D (2021) [211] ‡	L2P-M/D&C/LKH3	-	-	-	-	-	-	-	-	-	46.30	1.5s	65.20	38s
DIMES (2022) [180] ‡	L2P-O/MCTS	17.01	2.78%	3.1m	23.86	3.20%	3.6m	76.02	5.90%	33.7m	-	-	-	-
TSPformer (2023) [149] ‡	L2C	17.57	5.97%	3.1m	27.02	16.90%	5.0m	-	-	-	-	-	-	-
Pointerformer (2023) [141] ‡	L2C	17.14	3.56%	1.0m	24.80	7.30%	6.5m	-	-	-	-	-	-	-
H-TSP (2023) [71] ‡	L2C/D&C	-	-	-	24.65	6.62%	47s	77.75	7.32%	48s	-	-	-	-
TAM-AM (2023) [91] ‡	L2C/D&C/	-	-	-	-	-	-	-	-	-	50.06	0.8s	74.31	2.2s
TAM-LKH3 (2023) [91] ‡	L2P-O/D&C/LKH3	-	-	-	-	-	-	-	-	-	46.30	1.8s	64.80	5.6s
SO-DR (2023) [70]	L2P-M/D&C/LKH3	16.94	2.40%	-	23.77	2.80%	-	74.30	3.52%	-	-	-	-	-
UTSP (2023) [79]	L2P-O/BFLS	16.68	0.84%	-	23.39	1.17%	-	-	-	-	-	-	-	-
LEHD (2023) [44]	L2C/RRC	-	0.48%	-	-	1.22%	-	-	-	-	-	2.37%	-	-
DIFUSCO (2023) [53]	L2P-O/2-opt	16.63	0.46%	-	23.39	1.17%	-	73.62	2.58%	-	-	-	-	-
T2TCO (2023) [181]	L2P-O/2-opt	16.61	0.37%	-	23.30	0.78%	-	-	-	-	-	-	-	-
DeepACO (2023) [172] ‡	L2P-O/ACO	16.94	2.36%	4.3m	23.85	3.16%	1.1h	-	-	-	-	-	-	-
GLOP (2024) [162] ‡	L2I/D&C	16.91	1.99%	1.5m	23.84	3.11%	3.0m	75.29	4.90%	1.8m	-	-	-	-
GLOP-G (2024) [162] ‡	L2P-O/D&C/LKH3	-	-	-	-	-	-	-	-	-	45.90	1.1s	63.00	1.5s
UDC (2024) [77]	L2C/D&C	-	1.58%	-	-	1.78%	-	-	-	-	43.00	-	60.01	-
DEITSP (2025) [212]	L2P-O/2-opt	-	2.15%	-	-	3.68%	-	-	-	-	-	-	-	-
GELD (2025) [65]	L2C	-	0.52%	-	-	0.58%	-	-	2.38%	-	-	-	-	-

Note: Symbol ‡ denotes the results are taken from [162], and the rest are taken from the given references. For all experiments on TSP and CVRP, the terms "Obj." and "Gap" denote the average objective values and average gaps, respectively. The gaps are computed in comparison to the heuristic solver LKH3 [13] for both TSP and CVRP. In addition, following the prior study [162], "Time" denotes the overall and average running times on the corresponding TSP and CVRP test datasets, respectively. The TSP test datasets for 500-node and 1,000-node each consist of 128 instances, while the 10,000-node TSP test dataset includes 16 instances. The CVRP test datasets for 1,000-node and 2,000-node each consist of 100 instances.

For CVRP, a VRP variant beyond TSP, heatmap-based solvers lack the generality to handle it efficiently [50], [212]. Because the intricate capacity constraint of CVRP cannot be adequately modeled solely through the predicted heatmaps, as mentioned in Section V-C. We propose to employ more sophisticated OR algorithms (e.g., DP) for heatmap-based solvers, rather than relying solely on basic search algorithms. The incorporation of OR algorithms presents a viable direction to this problem. To solve large-scale CVRP instances, current NCO solvers generally adopt the D&C manner. For example, Zheng et al. [77] proposed a unified neural D&C framework (UDC), which employs an Anisotropic GNN for global node partitioning and subsequently adopts an L2C solver to solve all CVRP subproblems, achieving performance that surpasses LKH3.

Although certain solvers have attained strong performance on synthetic large-scale TSP and CVRP instances, their effectiveness on other VRP variants still requires improvement [210]. For instance, although UDC [77] employs the same L2C solver (i.e., ICAM [213]) for both CVRP and open VRP (OVRP), its performance on the 2000-node OVRP instances lags behind LKH3 by 8.71%. For future work in this regard, we suggest utilizing more lightweight NNs (e.g., Mamba [219]) as replacements for the widely adopted transformer in L2C solvers. This design adjustment enables L2C solvers to solve VRP with constraints from a global perspective, thereby mitigating the distance cost associated with inadequate decomposition strategies in the D&C manner. In addition, as mentioned in Section VI-A, there is a performance gap between existing solvers on synthetic datasets and real-world ones, which is an inadequacy that warrants further investigation in the future.

C. Inadequacies of Solving Complex VRP Variants

As shown in Section II-A, most NCO solvers specialize in TSP and CVRP, exhibiting inadequacies when faced with VRP incorporating other constraints [220], [221], [222]. Consequently, there arises a necessity to formulate solvers explicitly designed for VRP variants with intricate constraints in realistic scenarios [223]. For example, dispatching vehicle fleets within the premise of an airport can be considered a multiple-fleet VRP [6], encompassing various constraints such as capacity, time window, precedence, etc. In this section, we investigate methodologies for customizing NCO solvers to address the constraints posed by MOVRP and DVRP, two prominent VRP variants entailing constraints in real-world scenarios.

To tackle the problem arising from the multiple objectives in MOVRP, certain studies [117], [118], [119] adopted the MOEA/D framework [224]. This framework is designed to decompose a multiobjective problem into multiple distinct single-objective problems and develop models corresponding to each single-objective problem, thereby addressing them individually. Finally, the Pareto set is constructed based on solutions of all single-objective problems. Furthermore, Lin et al. [121] proposed an approach to approximate the Pareto set using a single preference-conditioned solver, instead of employing multiple solvers. This preference-conditioned solver takes preference vectors (each vector denotes a single-objective problem) as inputs and constructs all possible (sub)optimal solutions on the Pareto front. A critical factor in achieving high performance lies in the effective utilization of preference vectors, because improper handling may lead to gradient conflicts during training. In addition, certain studies have demonstrated the value of incorporating the imaging modality to address MOVRP. Chen et al. [225] proposed a graph-image bimodal fusion framework that enhances the performance of NCO by integrating graph and image information.

When dealing with DVRP, existing NCO solvers choose real-time embedding updates to meet the real-time requirements [130], [131], [132], [133], [134], [135], as mentioned in Section III-A. For example, Zhang et al. [130] proposed an L2C solver that handles a dynamic node pool. During each decoding step, this node pool is adjusted by removing or adding nodes to simulate canceled or new customer orders. Subsequently, they employed the updated node pool to update current embeddings. Moreover, in real-world scenarios, e.g., ride-hailing, NCO solvers encounter the dual challenge of dynamically updating nodes requiring visits while simultaneously adhering to time window constraints [132], [133], [223].

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:48:06 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Recently, several studies [76], [226], [227], [228], [229], [230] integrated multitask learning into the field of NCO, enabling solvers to adeptly tackle a range of VRP variants. For example, Liu et al. [226] incorporated the concept of compositional zero-shot learning to propose MTL, which formulates VRPs as different combinations of a set of constraints (e.g., capacity) and exploits a shared network to encode these constraints. To train the solver, MTL employs a joint loss function comprising multiple subloss functions of equal weightage, each corresponding to a distinct VRP variant. Zhou et al. [76] incorporated the concept of mixture-of-experts to further enhance the performance of MTL. However, learning multiple tasks simultaneously presents challenges not encountered in single-task learning, primarily due to potential conflicts between different task requirements [231], [232]. We advocate for alleviating conflicts by adjusting the weightage of all the subloss functions. For example, to better address this problem, Berto et al. [230] proposed mixed batch training and multivariant reward normalization. The former ensures that each training batch comprises VRP instances with diverse constraints, while the latter normalizes the reward for each VRP variant. In addition, Meta Learning offers a viable solution, which can discern relationships between tasks and dynamically adjust the weightages of their corresponding subloss functions. It is worth noting that certain studies [233], [234], [235] have also explored using a versatile solver to address different COPs, such as VRP and JSP, a goal that poses greater challenges than designing a unified solver for VRP variants.

Currently, these versatile solvers are limited to handling VRPs of sizes up to 100 and cannot effectively handle instances from different distributions. Future research should focus on enhancing their performance on large-scale VRP instances and across various distributions, while progressively incorporating additional VRP variants. For example, Goh et al. [236] recently proposed an NCO solver for multiple VRP variants that demonstrates robust out-of-distribution generalization. Once these powerful and versatile solvers have been developed to handle large-scale VRPs across diverse instance distributions and problem constraints, without the need for special designs and retraining, we anticipate the industry will be keen to adopt them in a wide range of applications.

D. Inadequacies of Comparison Methods

When comparing NCO solvers, fairness is only ensured if the gap is computed upon the same baseline, the test set sizes are equal, and the same hardware and parameters (e.g., the number of iterations) are utilized. However, the unfair comparison issue extends to a more significant issue when comparing NCO solvers with the conventional OR algorithms [74], [75], [237]. NCO solvers inherently benefit from operating on a GPU, whereas OR algorithms are traditionally executed on CPUs. Consequently, existing comparison methods unfairly favor NCO algorithms in terms of time, which results in the fact that most NCO solvers are still rarely adopted by the OR community [22].

For a fairer comparison between NCO solvers and OR algorithms, Liu et al. [22] simultaneously executed OR algorithms on multiple CPU threads to address multiple VRP instances. In addition, they introduced energy as a complementary metric in addition to performance gap and computation time. Specifically, energy denotes the electric power consumed by either NCO solvers or OR algorithms when solving test instances, recorded using the open-source PowerJoular tool [238]. In addition, Lu et al. [239] introduced the first robustness metric applicable to both OR algorithms and NCO solvers. This metric precisely measures the solver's cost value when dealing with postperturbation instances, i.e., the difference between the solution value obtained by the solver before and after the perturbation. Moreover, the postperturbation instance must ensure its optimal solution value does not deteriorate compared to the preperturbation state.

In the future, we foresee a practical approach aimed at normalizing computation time across different hardware scenarios for various NCO solvers and conventional OR algorithms. This approach involves applying suitable scaling factors to facilitate fair comparisons. In summary, establishing well-accepted benchmark libraries and guidelines to compare the results of NCO solvers with OR algorithms is crucial. These resources would enhance researchers' understanding of the strengths and weaknesses of NCO solvers, thereby promoting their broader adoption by the OR community.

VII. Conclusion

This article presents a systematic review to assess existing NCO solvers and achieves the following accomplishments while prior surveys did not, namely 1) all existing NCO solvers can be categorized accordingly with respect to our proposed taxonomy; 2) the inadequacies of existing NCO solvers, including generalization, and the resolution of large-scale and multiconstrained VRPs, as well as on-going efforts to solve these inadequacies are comprehensively introduced; 3) the advantages of SL and UL learning paradigms are well presented; and 4) the difficulties in applying NCO solvers in real-world VRPs are discussed. Through a comprehensive investigation, we identify open inadequacies for future research. Notably, ongoing efforts focus on only one or two of these inadequacies, with none attempting to address all of them. Furthermore, we propose numerous promising research directions to address these inadequacies. It must be acknowledged that due to the wide array of VRP variants, this survey solely delineates the inadequacies specific to the major variants. Nevertheless, this survey can alleviate the difficulties faced by fellow researchers dealing with nontraditional VRP variants and the other COPs. In addition, we provide a live web page repository for NCO solvers organized according to our proposed taxonomy, continuously updating with emerging NCO solvers. We anticipate that this survey and the live repository will achieve their intended objective of offering an overview of the latest developments in NCO solvers for VRPs, thereby serving as a valuable resource for researchers and practitioners.

At the same time, research on NCO solvers provides significant benefits to other scholars from related fields such as the broad machine learning and specific OR communities. For example, to reorder the optical character recognition (OCR) text blocks into a logical reading sequence, Li et al. [240] encoded each OCR text block as a node that encapsulates both visual layout and text semantic features, and subsequently employed a Ptr-Net-based network [31] to perform the reordering. To generate a shorter advertisement video, Tang et al. [241] have also drawn on NCO solvers and proposed M-SAN, which first segments the original video and subsequently selects and assembles certain segments into a shorter video. Furthermore, investigating the geometric invariance

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:48:06 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

properties of VRPs is also valuable for GNN-based point cloud analysis methods [242]. Advancements in NCO solvers will surely benefit these relevant machine learning studies. In the OR community, several studies (e.g., [51], [55], [172]) have successfully integrated NCO techniques into traditional OR algorithms, improving their efficiency. For example, the classic LKH algorithm [63] for solving VRP problems is computationally intensive because it requires a non-negligible number of iterations to obtain node penalties. Moreover, LKH relies on manually specified rules for constructing edge candidate sets, which could limit solution quality. To address this problem, Xin et al. [51] proposed NeuroLKH to simultaneously predict the edge candidate sets and node penalties. Specifically, NeuroLKH begins by encoding edge and node inputs into feature vectors and then employs two distinct decoders to predict edge scores and node penalties, respectively. We anticipate that future research will further exploit the complementary strengths of OR and NCO to address different optimization problems.

REFERENCES

[1] A. Schrijver, "On the history of combinatorial optimization (till 1960)," *Handb. Oper. Res. Manage. Sci.*, vol. 12, pp. 1–68, 2005.

[2] W. Yang, D. Wang, W. Pang, A.-H. Tan, and Y. Zhou, "Goods consumed during transit in split delivery vehicle routing problems: Modeling and solution," *IEEE Access*, vol. 8, pp. 110336–110350, 2020.

[3] X. Wu et al., "Incorporating surprisingly popular algorithm and Euclidean distance-based adaptive topology into PSO," *Swarm Evol. Comput.*, vol. 76, Feb. 2023, Art. no. 101222.

[4] Y. Ge, H. Li, and A. Tuzhilin, "Route recommendations for intelligent transportation services," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 3, pp. 1169–1182, Mar. 2021.

[5] Y. Xiao et al., "Distilling autoregressive models to obtain high-performance non-autoregressive solvers for vehicle routing problems with faster inference speed," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 20274–20283.

[6] J. Zhou, Y. Wu, Z. Cao, W. Song, J. Zhang, and Z. Chen, "Learning large neighborhood search for vehicle routing in airport ground handling," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 9, pp. 9769–9782, Sep. 2023.

[7] D. Kong, Y. Ma, Z. Cao, T. Yu, and J. Xiao, "Efficient neural collaborative search for pickup and delivery problems," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 11019–11034, Dec. 2024.

[8] K. Li et al., "Research reviews of combinatorial optimization methods based on deep reinforcement learning," *Acta Autom. Sinica*, vol. 47, no. 11, pp. 2521–2537, Nov. 2021.

[9] E. L. Lawler and D. E. Wood, "Branch-and-bound methods: A survey," *Operations Res.*, vol. 14, no. 4, pp. 699–719, Aug. 1966.

[10] R. Bellman, "On the theory of dynamic programming," *Proc. Nat. Acad. Sci. USA*, vol. 38, no. 8, pp. 716–719, 1952.

[11] P. Toth and D. Vigo, "Models, relaxations and exact approaches for the capacitated vehicle routing problem," *Discrete Appl. Math.*, vol. 123, no. 1, pp. 487–512, 2002.

[12] D. S. Hochba, "Approximation algorithms for NP-hard problems," *ACM Sigact News*, vol. 28, no. 2, pp. 40–52, 1997.

[13] K. Helsgaun, "An extension of the Lin-Kernighan-Helsgaun TSP solver for constrained traveling salesman and vehicle routing problems," *Roskilde, Roskilde Univ.*, vol. 12, pp. 966–980, 2017.

[14] X. Wu et al., "Neural architecture search for text classification with limited computing resources using efficient Cartesian genetic programming," *IEEE Trans. Evol. Comput.*, vol. 28, no. 3, pp. 638–652, Jun. 2024.

[15] W. Song et al., "Towards efficient few-shot graph neural architecture search via partitioning gradient contribution," in *Proc. ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2025, pp. 2609–2620.

[16] I. Boussaïd, J. Lepagnot, and P. Siarry, "A survey on optimization metaheuristics," *Inf. Sci.*, vol. 237, pp. 82–117, Jul. 2013.

[17] J. Sui et al.,A survey on deep learning-based algorithms for the traveling salesman problem,*Frontiers Comput. Sci.*, vol. 19, no. 6, Jun. 2025, Art. no. 196322.

[18] Y.-D. Kwon et al., "POMO: Policy optimization with multiple optima for reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 21188–21198.

[19] G. Ghiani et al., "Real-time vehicle routing: Solution concepts," *Eur. J. Oper. Res.*, vol. 151, no. 1, pp. 1–11, 2003.

[20] Z. Xu et al., "Large-scale order dispatch in on-demand ride-hailing platforms: A learning and planning approach," in *Proc. ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2018, pp. 905–913.

[21] Y. Bengio, A. Lodi, and A. Prouvost, "Machine learning for combinatorial optimization: A methodological tour d'horizon," *Eur. J. Oper. Res.*, vol. 290, no. 2, pp. 405–421, Apr. 2021.

[22] S. Liu, Y. Zhang, K. Tang, and X. Yao, "How good is neural combinatorial optimization? A systematic evaluation on the traveling salesman problem," *IEEE Comput. Intell. Mag.*, vol. 18, no. 3, pp. 14–28, Aug. 2023.

[23] Z. Zhang, J. Yang, Z. Cao, and H. C. Lau, "Neuro-ins: A learning-based one-shot node insertion for dynamic routing problems," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 9, pp. 5495–5507, Sep. 2025.

[24] M. Li et al., "Deep learning for approximate nearest neighbour search: A survey and future directions," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 9, pp. 8997–9018, Sep. 2023.

[25] N. Grinsztajn et al., "Winner takes it all: Training performant RL populations for combinatorial optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 48485–48509.

[26] R. Wang et al., "LinSATNet: The positive linear satisfiability neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 36605–36625.

[27] H. Liang et al., "SplitNet: A reinforcement learning based sequence splitting method for the minmax multiple travelling salesman problem," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 8720–8727.

[28] P. Xiao, Z. Zhang, J. Chen, J. Wang, and Z. Zhang, "Neural combinatorial optimization for robust routing problem with uncertain travel times," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 134841–134867.

[29] C. Zhang et al., "A review on learning to solve combinatorial optimisation problems in manufacturing," *IET Collaborative Intell. Manuf.*, vol. 5, no. 1, p. 12072, Mar. 2023.

[30] F. Chalumeau et al., "Combinatorial optimization with policy adaptation using latent space search," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 7947–7959.

[31] O. Vinyals, M. Fortunato, and N. Jaitly, "Pointer networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 2692–2700.

[32] W. Kool, H. V. Hoof, and M. Welling, "Attention, learn to solve routing problems!" in *Proc. Int. Conf. Learn. Represent.*, 2019, pp. 1–25.

[33] A. Bogyrbayeva, M. Meraliyev, T. Mustakhov, and B. Dauletbayev, "Machine learning to solve vehicle routing problems: A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 6, pp. 4754–4772, Jun. 2024.

[34] Q. Cappart et al., "Combinatorial optimization and reasoning with graph neural networks," *J. Mach. Learn. Res.*, vol. 24, no. 130, pp. 1–61, 2023.

[35] N. Mazyavkina et al., "Reinforcement learning for combinatorial optimization: A survey," *Comput. Oper. Res.*, vol. 134, 2021, Art. no. 105400.

[36] N. Parvez Farazi, B. Zou, T. Ahamed, and L. Barua, "Deep reinforcement learning in transportation research: A review," *Transp. Res. Interdiscipl. Perspect.*, vol. 11, Sep. 2021, Art. no. 100425.

[37] Q. Wang and C. Tang, "Deep reinforcement learning for transportation network combinatorial optimization: A survey," *Knowledge-Based Syst.*, vol. 233, Dec. 2021, Art. no. 107526.

[38] B. Li, G. Wu, Y. He, M. Fan, and W. Pedrycz, "An overview and experimental study of learning-based optimization algorithms for the vehicle routing problem," *IEEE/CAA J. Autom. Sinica*, vol. 9, no. 7, pp. 1115–1138, Jul. 2022.

[39] R. Shi and L. Niu, "A brief survey on learning based methods for vehicle routing problems," *Proc. Comput. Sci.*, vol. 221, pp. 773–780, Jan. 2023.

[40] K. T. Chung, C. K. M. Lee, and Y. P. Tsang, "Neural combinatorial optimization with reinforcement learning in industrial engineering: A survey," *Artif. Intell. Rev.*, vol. 58, no. 5, p. 130, Feb. 2025.

[41] D. Angioni, C. Archetti, and M. G. Speranza, "Neural combinatorial optimization: A tutorial," *Comput. Operations Res.*, vol. 182, Oct. 2025, Art. no. 107102.

[42] F. Zhou, A. Lischka, B. Kulcsár, J. Wu, M. Haghir Chehreghani, and G. Laporte, "Learning for routing: A guided review of recent developments and future directions," *Transp. Res. E, Logistics Transp. Rev.*, vol. 202, Oct. 2025, Art. no. 104278.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:48:06 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

[43] S. Yao, X. Lin, J. Wang, Q. Zhang, and Z. Wang, "Rethinking supervised learning-based neural combinatorial optimization for routing problem," *ACM Trans. Evol. Learn. Optim.*, vol. 5, no. 4, pp. 1–24, Dec. 2025.

[44] F. Luo, X. Lin, F. Liu, Q. Zhang, and Z. Wang, "Neural combinatorial optimization with heavy decoder: Toward large scale generalization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 8845–8864.

[45] C. K. Joshi, T. Laurent, and X. Bresson, "An efficient graph convolutional network technique for the travelling salesman problem," 2019, *arXiv:1906.01227*.

[46] Y. Xiao et al., "Reinforcement learning-based nonautoregressive solver for traveling salesman problems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 7, pp. 13402–13416, Jul. 2025.

[47] X. Chen and Y. Tian, "Learning to perform local rewriting for combinatorial optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 6278–6289.

[48] Y. Wu, W. Song, Z. Cao, J. Zhang, and A. Lim, "Learning improvement heuristics for solving routing problems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 9, pp. 5057–5069, Sep. 2022.

[49] P. R. D. O. da Costa et al., "Learning 2-opt heuristics for the traveling salesman problem via deep reinforcement learning," in *Proc. Asian Conf. Mach. Learn.*, vol. 129, 2020, pp. 465–480.

[50] Y. Ma, Z. Cao, and Y. M. Chee, "Learning to search feasible and infeasible regions of routing problems with flexible neural k-opt," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 49555–49578.

[51] L. Xin et al., "NeuroLKH: Combining deep learning model with Lin-Kernighan-Helsgaun heuristic for solving the traveling salesman problem," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 7472–7483.

[52] W. Kool et al., "Deep policy dynamic programming for vehicle routing problems," in *Proc. Int. Conf. Integr. Constraint Program., Artif. Intell. Oper. Res.*, 2022, pp. 190–213.

[53] Z. Sun and Y. Yang, "DIFUSCO: Graph-based diffusion solvers for combinatorial optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 3706–3731.

[54] A. Hottung, B. Bhandari, and K. Tierney, "Learning a latent search space for routing problems using variational autoencoders," in *Proc. Int. Conf. Learn. Represent.*, 2021, pp. 1–13.

[55] J. Zheng, K. He, J. Zhou, Y. Jin, and C.-M. Li, "Combining reinforcement learning with lin-kernighan-helsgaun algorithm for the traveling salesman problem," in *Proc. AAAI Conf. Artif. Intell.*, May 2021, vol. 35, no. 14, pp. 12445–12452.

[56] T. Dernedde et al., "MOCO: A learnable meta optimizer for combinatorial optimization," in *Proc. Adv. Knowl. Discovery Data Mining*, 2025, pp. 236–248.

[57] F. Liu et al., "Evolution of heuristics: Towards efficient automatic algorithm design using large language model," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 32201–32223.

[58] X. Bresson and T. Laurent, "The transformer network for the traveling salesman problem," 2021, *arXiv:2103.03012*.

[59] L. Xin et al., "Multi-decoder attention model with embedding glimpse for solving vehicle routing problems," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 12042–12049.

[60] G. Schrimpf, J. Schneider, H. Stamm-Wilbrandt, and G. Dueck, "Record breaking optimization results using the ruin and recreate principle," *J. Comput. Phys.*, vol. 159, no. 2, pp. 139–171, Apr. 2000.

[61] A. Hottung and K. Tierney, "Neural large neighborhood search for routing problems," *Artif. Intell.*, vol. 313, 2022, Art. no. 103786.

[62] M. Kim, J. Park, and J. Park, "Learning to cross exchange to solve min-max vehicle routing problems," in *Proc. Int. Conf. Learn. Represent.*, 2023, pp. 1–29.

[63] K. Helsgaun, "An effective implementation of the Lin–Kernighan traveling salesman heuristic," *Eur. J. Oper. Res.*, vol. 126, no. 1, pp. 106–130, Oct. 2000.

[64] Y. Hu, Y. Yao, J. Chen, Z. Wang, Q. Jia, and Y. Pan, "Solving scalable multiagent routing problems with reinforcement learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 11, pp. 19604–19618, Nov. 2025.

[65] Y. Xiao et al., "GELD: A unified neural model for efficiently solving traveling salesman problems across different scales," *Pattern Recognit.*, vol. 173, 2026, Art. no. 112865.

[66] C. K. Joshi, Q. Cappart, L.-M. Rousseau, and T. Laurent, "Learning the travelling salesperson problem requires rethinking generalization," *Constraints*, vol. 27, nos. 1–2, pp. 70–98, Apr. 2022.

[67] J. Bi et al., "Learning generalizable models for vehicle routing problems via knowledge distillation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 31226–31238.

[68] Y. Jiang, Y. Wu, Z. Cao, and J. Zhang, "Learning to solve routing problems via distributionally robust optimization," in *Proc. AAAI Conf. Artif. Intell.*, Jun. 2022, vol. 36, no. 9, pp. 9786–9794.

[69] H. Fang et al., "INViT: A generalizable routing problem solver with invariant nested view transformer," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 12973–12992.

[70] H. Cheng et al., "Select and optimize: Learning to solve large-scale TSP instances," in *Proc. Int. Conf. Artif. Intell. Stat*, 2023, pp. 1219–1231.

[71] X. Pan et al., "H-TSP: Hierarchically solving the large-scale traveling salesman problem," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 9343–9353.

[72] S. Liu, X. Yan, and Y. Jin, "An edge-aware graph autoencoder trained on scale-imbalanced data for traveling salesman problems," *Knowl.-Based Syst.*, vol. 291, May 2024, Art. no. 111559.

[73] Y. D. Kwon, J. Choo, I. Yoon, M. Park, D. Park, and Y. Gwon, "Matrix encoding networks for neural combinatorial optimization," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 7, 2021, pp. 5138–5149.

[74] A. I. Garmendia, J. Ceberio, and A. Mendiburu, "Applicability of neural combinatorial optimization: A critical view," *ACM Trans. Evol. Learn. Optim.*, vol. 4, no. 3, pp. 1–26, Sep. 2024.

[75] L. Accorsi, A. Lodi, and D. Vigo, "Guidelines for the computational testing of machine learning approaches to vehicle routing problems," *Operations Res. Lett.*, vol. 50, no. 2, pp. 229–234, Mar. 2022.

[76] J. Zhou et al., "MVMoE: Multi-task vehicle routing solver with mixture-of-experts," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 61804–61824.

[77] Z. Zheng, C. Zhou, X. Tong, M. Yuan, and Z. Wang, "UDC: A unified neural divide-and-conquer framework for large-scale combinatorial optimization problems," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 6081–6125.

[78] M. Panzer and B. Bender, "Deep reinforcement learning in production systems: A systematic literature review," *Int. J. Prod. Res.*, vol. 60, no. 13, pp. 4316–4341, Jul. 2022.

[79] Y. Min, Y. Bai, and C. Gomes, "Unsupervised learning for solving the travelling salesman problem," in *Proc. Adv. Neural Inf. Process. Syst.* 36, 2023, pp. 47264–47278.

[80] C. Zhang, W. Song, Z. Cao, J. Zhang, P. S. Tan, and C. Xu, "Learning to dispatch for job shop scheduling via deep reinforcement learning," in *Proc. Neural Inf. Process. Syst. (NeurIPS)*, 2020, pp. 1621–1632.

[81] F. Ni et al., "A multi-graph attributed reinforcement learning based optimization algorithm for large-scale hybrid flow shop scheduling problem," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2021, pp. 3441–3451.

[82] C. Zhang et al., "Deep reinforcement learning guided improvement heuristic for job shop scheduling," in *Proc. Int. Conf. Learn. Represent.*, 2024, pp. 1–20.

[83] P. Zhao et al., "Dual operation aggregation graph neural networks for solving flexible job-shop scheduling problem with reinforcement learning," in *Proc. ACM Web Conf.*, Apr. 2025, pp. 4089–4100.

[84] H. Zhao and K. Xu, "Learning efficient online 3D bin packing on packing configuration trees," in *Proc. Int. Conf. Learn. Represent.*, 2022, pp. 1–18.

[85] Y. Jiang, Z. Cao, and J. Zhang, "Learning to solve 3-D bin packing problem via deep reinforcement learning and constraint programming," *IEEE Trans. Cybern.*, vol. 53, no. 5, pp. 2864–2875, May 2023.

[86] H. Cheng et al., "Learning to solve class-constrained bin packing problems via encoder–decoder model," in *Proc. Int. Conf. Learn. Represent.*, 2024, pp. 1–24.

[87] Y. Min and C. P. Gomes, "On size and hardness generalization in unsupervised learning for the travelling salesman problem," 2024, *arXiv:2403.20212*.

[88] C. E. Miller, A. W. Tucker, and R. A. Zemlin, "Integer programming formulation of traveling salesman problems," *J. ACM*, vol. 7, no. 4, pp. 326–329, Oct. 1960.

[89] Y. Ma et al., "Learning to iteratively solve routing problems with dual-aspect collaborative transformer," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 11096–11107.

[90] M. Kim, J. Park, and J. Park, "Sym-NCO: Leveraging symmetricity for neural combinatorial optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 1936–1949.

[91] Q. Hou et al., "Generalize learned heuristics to solve large-scale vehicle routing problems in real-time," in *Proc. Int. Conf. Learn. Represent.*, 2023, pp. 1–37.

[92] I. Bello, "Neural combinatorial optimization with reinforcement learning," in *Proc. Int. Conf. Learn. Represent. (Workshop)*, 2017, pp. 1–5.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:48:06 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

[93] E. Khalil et al., "Learning combinatorial optimization algorithms over graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6351–6361.

[94] M. Nazari, A. Oroojlooy, M. Takáč, and L. V. Snyder, "Reinforcement learning for solving the vehicle routing problem," in *Proc. 32nd Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 9861–9871.

[95] L. Xin, W. Song, Z. Cao, and J. Zhang, "Step-wise deep learning models for solving routing problems," *IEEE Trans. Ind. Informat.*, vol. 17, no. 7, pp. 4861–4871, Jul. 2021.

[96] M. Kim, J. Park, and J. Kim, "Learning collaborative policies to solve NP-hard routing problems," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 10418–10430.

[97] Y. Xu, M. Fang, L. Chen, G. Xu, Y. Du, and C. Zhang, "Reinforcement learning with multiple relational attention for solving vehicle routing problems," *IEEE Trans. Cybern.*, vol. 52, no. 10, pp. 11107–11120, Oct. 2022.

[98] A. Hottung, Y.-D. Kwon, and K. Tierney, "Efficient active search for combinatorial optimization problems," in *Proc. Int. Conf. Learn. Represent.*, 2022, pp. 1–13.

[99] D. Drakulic, S. Michel, F. Mai, A. Sors, and J.-M. Andreoli, "BQ-NCO: Bisimulation quotienting for efficient neural combinatorial optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 77416–77429.

[100] J. Son et al., "Meta-SAGE: Scale meta-learning scheduled adaptation with guided exploration for mitigating scale shift on combinatorial optimization," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 32194–32210.

[101] R. Sun, Z. Zheng, and Z. Wang, "Learning encodings for constructive neural combinatorial optimization needs to regret," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 20803–20811.

[102] T. Q. T. Vo et al., "Learning to cut generation in branch-and-cut algorithms for combinatorial optimization," *ACM Trans. Evol. Learn. Optim.*, vol. 5, no. 3, pp. 1–27, 2025.

[103] W. Ouyang, Y. Wang, P. Weng, and S. Han, "Generalization in deep RL for TSP problems via equivariance and local search," *Social Netw. Comput. Sci.*, vol. 5, no. 4, p. 369, Mar. 2024.

[104] A. Hottung and K. Tierney, "Neural large neighborhood search for the capacitated vehicle routing problem," in *Proc. Eur. Conf. Artif. Intell.*, 2020, pp. 1–8.

[105] H. Lu, X. Zhang, and S. Yang, "A learning-based iterative method for solving vehicle routing problems," in *Proc. Int. Conf. Learn. Represent.*, 2020, pp. 1–15.

[106] J. Zhao, M. Mao, X. Zhao, and J. Zou, "A hybrid of deep reinforcement learning and local search for the vehicle routing problems," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 11, pp. 7208–7218, Nov. 2021.

[107] K. Zhang, "Multi-vehicle routing problems with soft time windows: A multi-agent reinforcement learning approach," *Transp. Res. C, Emerg. Tech.*, vol. 121, 2020, Art. no. 102861.

[108] R. Zhang et al., "Learning to solve multiple-TSP with time window and rejections via deep reinforcement learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 1, pp. 1325–1336, Jan. 2023.

[109] B. Lin, B. Ghaddar, and J. Nathwani, "Deep reinforcement learning for the electric vehicle routing problem with time windows," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 8, pp. 11528–11538, Aug. 2022.

[110] C. Chi et al., "A deep reinforcement learning framework for column generation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 9633–9644.

[111] Q. Ma et al., "Combinatorial optimization by graph pointer networks and hierarchical reinforcement learning," in *Proc. AAAI Conf. Artif. Intell. (Workshop)*, 2020, pp. 1–8.

[112] J. K. Falkner and L. Schmidt-Thieme, "Learning to solve vehicle routing problems with time windows through joint attention," 2020, *arXiv:2006.09100*.

[113] M. A. L. Silva, "A reinforcement learning-based multi-agent framework applied for solving routing and scheduling problems," *Expert Syst. Appl.*, vol. 131, pp. 148–171, 2019.

[114] C. Dang et al., "Warm-starting nested rollout policy adaptation with optimal stopping," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 12381–12389.

[115] J. Chen et al., "Looking ahead to avoid being late: Solving hard-constrained traveling salesman problem," in *Proc. 6th Int. Conf. Distrib. Artif. Intelligences*, Dec. 2024, pp. 1–12.

[116] J. Bi et al., "Learning to handle complex constraints for vehicle routing problems," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 93479–93509.

[117] H. Wu, J. Wang, and Z. Zhang, "MODRL/D-AM: Multiobjective deep reinforcement learning algorithm using decomposition and attention model for multiobjective optimization," in *Proc. Int. Symp. Intell. Comput. Appl.*, 2020, pp. 575–589.

[118] K. Li, T. Zhang, and R. Wang, "Deep reinforcement learning for multiobjective optimization," *IEEE Trans. Cybern.*, vol. 51, no. 6, pp. 3103–3114, Jun. 2021.

[119] Z. Zhang, Z. Wu, H. Zhang, and J. Wang, "Meta-learning-based deep reinforcement learning for multiobjective optimization problems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 10, pp. 7978–7991, Oct. 2023.

[120] X. Cai et al., "The collaborative local search based on dynamic-constrained decomposition with grids for combinatorial multiobjective optimization," *IEEE Trans. Cybern.*, vol. 51, no. 5, pp. 2639–2650, May 2021.

[121] X. Lin, Z. Yang, and Q. Zhang, "Pareto set learning for neural multi-objective combinatorial optimization," in *Proc. Int. Conf. Learn. Represent.*, 2022, pp. 1–31.

[122] J. Chen, "Efficient meta neural heuristic for multi-objective combinatorial optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 56825–56837.

[123] J. Chen et al., "Neural multi-objective combinatorial optimization with diversity enhancement," in *Proc. Adv. Neural Inf. Process. Syst.* 36, 2023, pp. 39176–39188.

[124] Z. Wang, S. Yao, G. Li, and Q. Zhang, "Multiobjective combinatorial optimization using a single deep reinforcement learning model," *IEEE Trans. Cybern.*, vol. 54, no. 3, pp. 1984–1996, Mar. 2024.

[125] G. Luo and J. Luo, "Multi-objective adaptive dynamics attention model to solve multi-objective vehicle routing problem," in *Proc. Asian Conf. Mach. Learn.*, 2024, pp. 834–849.

[126] Y. Lu et al., "Context-aware diversity enhancement for neural multi-objective combinatorial optimization," 2024, *arXiv:2405.08604*.

[127] M. Fan et al., "Conditional neural heuristic for multiobjective vehicle routing problems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 3, pp. 4677–4689, Mar. 2025.

[128] X. Li et al., "Learning to optimize industry-scale dynamic pickup and delivery problems," in *Proc. IEEE 37th Int. Conf. Data Eng. (ICDE)*, Apr. 2021, pp. 2511–2522.

[129] Y. Ma et al., "A hierarchical reinforcement learning based optimization framework for large-scale dynamic pickup and delivery problems," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 23609–23620.

[130] Z. Zhang, H. Liu, M. Zhou, and J. Wang, "Solving dynamic traveling salesman problems with deep reinforcement learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 4, pp. 2119–2132, Apr. 2023.

[131] N. Paul, "Multi-agent neural rewriter for vehicle routing with limited disclosure of costs," in *Proc. Int. Conf. Learn. Represent. (workshop)*, 2022, pp. 1–17.

[132] A. Sivagnanam et al., "Offline vehicle routing problem with online bookings: A novel problem formulation with applications to paratransit," in *Proc. Int. Joint Conf. Artif. Intell.*, Jul. 2022, pp. 3933–3939.

[133] W. Joe and H. C. Lau, "Deep reinforcement learning approach to solve dynamic vehicle routing problem with stochastic customers," in *Proc. Int. Conf. Autom. Plan. Sched.*, 2020, pp. 394–402.

[134] J. J. Q. Yu, W. Yu, and J. Gu, "Online vehicle routing with neural combinatorial optimization and deep reinforcement learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 20, no. 10, pp. 3806–3817, Oct. 2019.

[135] W. Pan and S. Q. Liu, "Deep reinforcement learning for the dynamic and uncertain vehicle routing problem," *Int. J. Speech Technol.*, vol. 53, no. 1, pp. 405–422, Jan. 2023.

[136] J. Li, L. Xin, Z. Cao, A. Lim, W. Song, and J. Zhang, "Heterogeneous attentions for solving pickup and delivery problem via deep reinforcement learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 3, pp. 2306–2315, Mar. 2022.

[137] Y. Ma et al., "Efficient neural neighborhood search for pickup and delivery problems," in *Proc. Int. Joint Conf. Artif. Intell.*, Jul. 2022, pp. 4776–4784.

[138] Z. Zong et al., "MAPDP: Cooperative multi-agent reinforcement learning to solve pickup and delivery problems," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 9980–9988.

[139] K. J. Chaitanya, L. Thomas, and B. Xavier, "On learning paradigms for the travelling salesman problem," in *Proc. Adv. Neural Inf. Process. Syst. (Workshop)*, 2019, pp. 1–9.

[140] J. Zheng, K. He, J. Zhou, Y. Jin, and C.-M. Li, "Reinforced lin–kernighan–helsgaun algorithms for the traveling salesman problems," *Knowl.-Based Syst.*, vol. 260, Jan. 2023, Art. no. 110144.

[141] Y. Jin et al., "Pointerformer: Deep reinforced multi-pointer transformer for the traveling salesman problem," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 8132–8140.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:48:06 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

[142] Z. Zheng et al., "DPN: Decoupling partition and navigation for neural solvers of min-max vehicle routing problems," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 61559–61592.

[143] J. Son et al., "Equity-Transformer: Solving NP-hard min-max routing problems as sequential generation with equity context," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 20265–20273.

[144] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Mach. Learn.*, vol. 8, nos. 3–4, pp. 229–256, May 1992.

[145] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.

[146] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.

[147] Z. Yuan, G. Li, Z. Wang, J. Sun, and R. Cheng, "RL-CSL: A combinatorial optimization method using reinforcement learning and contrastive self-supervised learning," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 7, no. 4, pp. 1010–1024, Aug. 2023.

[148] A. Hottung, M. Mahajan, and K. Tierney, "PolyNet: Learning diverse solution strategies for neural combinatorial optimization," in *Proc. Int. Conf. Learn. Represent.*, 2025, pp. 1–19.

[149] H. Yang, M. Zhao, L. Yuan, Y. Yu, Z. Li, and M. Gu, "Memory-efficient transformer-based network model for traveling salesman problem," *Neural Netw.*, vol. 161, pp. 589–597, Apr. 2023.

[150] A. N. Gomez et al., "The reversible residual network: Backpropagation without storing activations," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 2211–2221.

[151] H. Zhou et al., "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 11106–11115.

[152] K. Li, T. Zhang, R. Wang, Y. Wang, Y. Han, and L. Wang, "Deep reinforcement learning for combinatorial optimization: Covering salesman problems," *IEEE Trans. Cybern.*, vol. 52, no. 12, pp. 13142–13155, Dec. 2022.

[153] J. Li et al., "Learning feature embedding refiner for solving vehicle routing problems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 11, pp. 15279–15291, Nov. 2024.

[154] A. Nowak, D. Folqué, and J. Bruna, "Divide and conquer networks," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–15.

[155] D. L. Applegate,*The traveling salesman problem: A computational study*, in *The Traveling Salesman Problem*. Princeton, NJ, USA: Princeton Univ. Press, 2011.

[156] T. Vidal, "Hybrid genetic search for the CVRP: Open-source implementation and swap neighborhood," *Comput. Oper. Res.*, vol. 140, 2022, Art. no. 105643.

[157] J. Choo et al., "Simulation-guided beam search for neural combinatorial optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 8760–8772.

[158] F. J. C. Verdú, L. Castelli, and L. Bortolussi, "Scaling combinatorial optimization neural improvement heuristics with online search and adaptation," in *Proc. AAAI Conf. Artif. Intell.*, 2025, pp. 27135–27143.

[159] M. Kim et al., "Scale-conditioned adaptation for large scale combinatorial optimization," in *Proc. Adv. Neural Inf. Process. Syst. (Workshop)*, 2022, pp. 1–9.

[160] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. learn. Represent.*, 2022, pp. 1–26.

[161] J. Sui et al., "Learning 3-opt heuristics for traveling salesman problem via deep reinforcement learning," in *Proc. Asian Conf. Mach. Learn.*, vol. 157, 2021, pp. 1301–1316.

[162] H. Ye et al., "GLOP: Learning global partition and local construction for solving large-scale routing problems in real-time," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 20284–20292.

[163] J. Redmon et al., "You only look once: Unified," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 779–788.

[164] K. Li et al., "Destroy and repair using hyper-graphs for routing," in *Proc. AAAI Conf. Artif. Intell.*, 2025, pp. 18341–18349.

[165] É. D. Taillard and K. Helsgaun, "POPMUSIC for the travelling salesman problem," *Eur. J. Oper. Res.*, vol. 272, no. 2, pp. 420–429, Jan. 2019.

[166] M. Kim et al., "Ant colony sampling with GFlownets for combinatorial optimization," in *Proc. Int. Conf. Artif. Intell. Stat.*, 2025, pp. 469–477.

[167] Y. Sun, A. Ernst, X. Li, and J. Weiner, "Generalization of machine learning for problem reduction: A case study on travelling salesman problems," *OR Spectr.*, vol. 43, no. 3, pp. 607–633, Sep. 2021.

[168] M. Prates et al., "Learning to solve NP-complete problems: A graph neural network for decision TSP," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 4731–4738.

[169] Y. Sun, X. Li, and A. Ernst, "Using statistical measures and machine learning for graph reduction to solve maximum weight clique problems," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 5, pp. 1746–1760, May 2021.

[170] B. Hudson et al., "Graph neural network guided local search for the traveling salesperson problem," in *Proc. Int. Conf. Learn. Represent.*, 2022, pp. 1–20.

[171] Y. Liang Goh, W. Sun Lee, X. Bresson, T. Laurent, and N. Lim, "Combining reinforcement learning and optimal transport for the traveling salesman problem," 2022, *arXiv:2203.00903*.

[172] H. Ye, J. Wang, Z. Cao, H. Liang, and Y. Li, "DeepACO: Neural-enhanced ant systems for combinatorial optimization," in *Proc. Adv. Neural Inf. Process. Syst.* 36, 2023, pp. 43706–43728.

[173] M. Karimi-Mamaghan, M. Mohammadi, P. Meyer, A. M. Karimi-Mamaghan, and E.-G. Talbi, "Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: A state-of-the-art," *Eur. J. Oper. Res.*, vol. 296, no. 2, pp. 393–422, Jan. 2022.

[174] P. Kerschke, L. Kotthoff, J. Bossek, H. H. Hoos, and H. Trautmann, "Leveraging TSP solver complementarity through machine learning," *Evol. Comput.*, vol. 26, no. 4, pp. 597–620, Dec. 2018.

[175] A. E. Gutierrez-Rodríguez, S. E. Conant-Pablos, J. C. Ortiz-Bayliss, and H. Terashima-Marín, "Selecting meta-heuristics for solving vehicle routing problems with time windows via meta-learning," *Expert Syst. Appl.*, vol. 118, pp. 470–481, Mar. 2019.

[176] W. Qin, Z. Zhuang, Z. Huang, and H. Huang, "A novel reinforcement learning-based hyper-heuristic for heterogeneous vehicle routing problem," *Comput. Ind. Eng.*, vol. 156, Jun. 2021, Art. no. 107252.

[177] F. Yang, T. Jin, T.-Y. Liu, X. Sun, and J. Zhang, "Boosting dynamic programming with neural networks for solving NP-hard problems," in *Proc. Asian Conf. Mach. Learn.*, 2018, pp. 726–739.

[178] S. Xu, S. S. Panwar, M. Kodialam, and T. Lakshman, "Deep neural network approximated dynamic programming for combinatorial optimization," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 2, 2020, pp. 1684–1691.

[179] Z.-H. Fu, K.-B. Qiu, and H. Zha, "Generalize a small pre-trained model to arbitrarily large TSP instances," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 7474–7482.

[180] R. Qiu, Z. Sun, and Y. Yang, "DIMES: A differentiable meta solver for combinatorial optimization problems," in *Proc. Adv. Neural Inf. Process. Syst.* 35, 2022, pp. 25531–25546.

[181] Y. Li, J. Guo, R. Wang, and J. Yan, "T2T: From distribution learning in training to gradient search in testing for combinatorial optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 50020–50040.

[182] H. Ye et al., "ReEvo: Large language models as hyper-heuristics with reflective evolution," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 43571–43608.

[183] X. Wu et al., "Efficient heuristics generation for solving combinatorial optimization problems using large language models," in *Proc. 31st ACM SIGKDD Conf. Knowl. Discovery Data Mining V.2*, Aug. 2025, pp. 3228–3239.

[184] C. Voudouris and E. Tsang, "Guided local search and its application to the traveling salesman problem," *Eur. J. Oper. Res.*, vol. 113, no. 2, pp. 469–499, Mar. 1999.

[185] F. Xie, M. Müller, and R. Holte, "Adding local exploration to greedy best-first search in satisficing planning," in *Proc. AAAI Conf. Artif. Intell.*, 2014, pp. 2388–2394.

[186] S. Das, A. Abraham, U. K. Chakraborty, and A. Konar, "Differential evolution using a neighborhood-based mutation operator," *IEEE Trans. Evol. Comput.*, vol. 13, no. 3, pp. 526–553, Jun. 2009.

[187] F. Liu, X. Tong, M. Yuan, and Q. Zhang, "Algorithm evolution using large language model," 2023, *arXiv:2311.15249*.

[188] Y. Wang, Y.-H. Jia, W.-N. Chen, and Y. Mei, "Distance-aware attention reshaping for enhancing generalization of neural solvers," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 10, pp. 18900–18914, Oct. 2025.

[189] A. François, Q. Cappart, and L.-M. Rousseau, "How to evaluate machine learning approaches for combinatorial optimization: Application to the travelling salesman problem," 2019, *arXiv:1909.23121*.

[190] C. Caramanis, D. Fotakis, A. Kalavasis, V. Kontonis, and C. Tzamos, "Optimizing solution-samplers for combinatorial problems: The landscape of policy-gradient method," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 14035–14069.

[191] H. Sun et al., "Revisiting sampling for combinatorial optimization," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 32859–32874.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:48:06 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

[192] C. Gao et al., "Towards generalizable neural solvers for vehicle routing problems via ensemble with transferrable local policy," in *Proc. Int. Joint Conf. Artif. Intell.*, 2024, pp. 6914–6922.

[193] A. Bdeir, J. K. Falkner, and L. Schmidt-Thieme, "Attention, filling in the gaps for generalization in routing problems," in *Proc. Mach. Learn. Knowl. Discovery Databas.*, 2022, pp. 505–520.

[194] N. Sultana, J. Chan, T. Sarwar, and A. K. Qin, "Learning to optimise general TSP instances," *Int. J. Mach. Learn. Cybern.*, vol. 13, no. 8, pp. 2213–2228, Aug. 2022.

[195] Y. Jiang, Z. Cao, Y. Wu, W. Song, and J. Zhang, "Ensemble-based deep reinforcement learning for vehicle routing problems under distribution shift," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 53112–53125.

[196] C. Wang, Z. Yu, S. McAleer, T. Yu, and Y. Yang, "ASP: Learn a universal neural solver!" *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 6, pp. 4102–4114, Jun. 2024.

[197] F. Liu et al., "Prompt learning for generalized vehicle routing," in *Proc. Int. Joint Conf. Artif. Intell.*, 2024, pp. 6976–6984.

[198] J. Zhou, Y. Wu, Z. Cao, W. Song, J. Zhang, and Z. Shen, "Collaboration! Towards robust neural methods for routing problems," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 121731–121764.

[199] Z. Zhang et al., "Learning to solve travelling salesman problem with hardness-adaptive curriculum," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 9136–9144.

[200] S. Geisler et al., "Generalization of neural combinatorial solvers through the lens of adversarial robustness," in *Proc. Int. Conf. Learn. Represent.*, 2022, pp. 1–21.

[201] Y. Jiang et al., "Multi-view graph contrastive learning for solving vehicle routing problems," in *Proc. Conf. Uncertainty Artif. Intell.*, 2023, pp. 984–994.

[202] S. Manchanda et al., "On the generalization of neural combinatorial optimization heuristics," in *Proc. Mach. Learn. Knowl. Discovery Databases*, 2022, pp. 426–442.

[203] J. Zhou et al., "Towards omni-generalizable neural methods for vehicle routing problems," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 42769–42789.

[204] Y. Xiao et al., "Improving generalization of neural vehicle routing problem solvers through the lens of model architecture," *Neural Netw.*, vol. 187, Jul. 2025, Art. no. 107380.

[205] T. Q. T. Vo et al., "Improving subtour elimination constraint generation in branch-and-cut algorithms for the tsp with machine learning," in *Proc. Learn. Intell. Optimiz.*, 2023, pp. 537–551.

[206] R. Bai et al., "Analytics and machine learning in vehicle routing research," *Int. J. Prod. Res.*, vol. 61, no. 1, pp. 4–30, 2023.

[207] W. Ouyang, Y. Wang, S. Han, Z. Jin, and P. Weng, "Improving generalization of deep reinforcement learning-based TSP solvers," in *Proc. IEEE Symp. Ser. Comput. Intell. (SSCI)*, Dec. 2021, pp. 01–08.

[208] D. Zhang et al., "Neural TSP solver with progressive distillation," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 12147–12154.

[209] X. Chen et al., "ExtNCO: A fine-grained divide-and-conquer approach for extending NCO to solve large-scale traveling salesman problem," *Expert Syst. Appl.*, vol. 286, 2025, Art. no. 127950.

[210] X. Wu et al., "Efficient neural combinatorial optimization solver for the min-max heterogeneous capacitated vehicle routing problem," *Expert Syst. Appl.*, vol. 331, Dec. 2026, Art. no. 133234.

[211] S. Li, Z. Yan, and C. Wu, "Learning to delegate for large-scale vehicle routing," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 26198–26211.

[212] M. Wang et al., "An efficient diffusion-based non-autoregressive solver for traveling salesman problem," in *Proc. ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2025, pp. 1469–1480.

[213] C. Zhou et al., "ICAM: Rethinking instance-conditioned adaptation in neural vehicle routing solver," *IEEE Trans. Intell. Transp. Syst.*, vol. 27, no. 7, pp. 8270–8284, 2026.

[214] J. Pirnay and D. G. Grimm, "Self-improvement for neural combinatorial optimization: Sample without replacement, but improvement," *Trans. Mach. Learn. Res.*, pp. 1–30, 2024.

[215] F. Luo et al., "Boosting neural combinatorial optimization for large-scale vehicle routing problems," in *Proc. Int. Conf. Learn. Represent.*, 2025, pp. 1–27.

[216] Y. Xiao et al., "DGL: Dynamic global-local information aggregation for scalable VRP generalization with self-improvement learning," in *Proc. Thirty-ThirdInternational Joint Conf. Artif. Intell.*, Aug. 2024, pp. 8669–8677.

[217] Y. Li, J. Guo, R. Wang, H. Zha, and J. Yan, "Fast T2T: Optimization consistency speeds up diffusion-based training-to-testing solving for combinatorial optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 30179–30206.

[218] Y. Lipman et al., "Flow matching for generative modeling," in *Proc. Int. Conf. Learn. Represent.*, 2023, pp. 1–28.

[219] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," in *Proc. Conf. Lang. Model.*, 2024, pp. 1–32.

[220] V.-A. Darvariu, S. Hailes, and M. Musolesi, "Graph reinforcement learning for combinatorial optimization: A survey and unifying perspective," *Trans. Mach. Learn. Res.*, pp. 1–46, 2024.

[221] L. Luttmann and L. Xie, "Neural combinatorial optimization on heterogeneous graphs. An application to the picker routing problem in mixed-shelves warehouses," in *Proc. Int. Conf. Autom. Plan. Sched.*, 2024, pp. 1–9.

[222] C. Wang, Z. Cao, Y. Wu, L. Teng, and G. Wu, "Deep reinforcement learning for solving vehicle routing problems with backhauls," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 3, pp. 4779–4793, Mar. 2025.

[223] Y. Tong, D. Shi, Y. Xu, W. Lv, Z. Qin, and X. Tang, "Combinatorial optimization meets reinforcement learning: Effective taxi order dispatching at large-scale," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 10, pp. 9812–9823, Oct. 2023.

[224] Q. Zhang and H. Li, "MOEA/D: A multiobjective evolutionary algorithm based on decomposition," *IEEE Trans. Evol. Comput.*, vol. 11, no. 6, pp. 712–731, Dec. 2007.

[225] J. Chen et al., "Neural multi-objective combinatorial optimization via graph-image multimodal fusion," in *Proc. Int. Conf. Learn. Represent.*, 2025, pp. 1–20.

[226] F. Liu, X. Lin, Z. Wang, Q. Zhang, T. Xialiang, and M. Yuan, "Multi-task learning for routing problem with cross-problem zero-shot generalization," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2024, pp. 1898–1908.

[227] C. Wang and T. Yu, "Efficient training of multi-task combinatorial neural solver with multi-armed bandits," *Trans. Mach. Learn. Res.*, pp. 1–31, 2025.

[228] Z. Lin et al., "Cross-problem learning for solving vehicle routing problems," in *Proc. Int. Joint Conf. Artif. Intell.*, 2024, pp. 6958–6966.

[229] H. Li et al., "CaDA: Cross-problem routing solver with constraint-aware dual-attention," in *Proc. Int. Conf. Mach. Learn.*, 2025, pp. 35438–35456.

[230] F. Berto et al., "Routefinder: Towards foundation models for vehicle routing problems," *Trans. Mach. Learn. Res.*, pp. 1–41, 2025.

[231] Y. Zhang and Q. Yang, "A survey on multi-task learning," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 12, pp. 5586–5609, Dec. 2022.

[232] M. Crawshaw, "Multi-task learning with deep neural networks: A survey," 2020, *arXiv:2009.09796*.

[233] D. Drakulic, S. Michel, and J.-M. Andreoli, "GOAL: A generalist combinatorial optimization agent learner," in *Proc. Int. Conf. Learn. Represent.*, 2025, pp. 1–24.

[234] W. Pan et al., "UniCO: On unified combinatorial optimization via problem reduction to matrix-encoded general TSP," in *Proc. Int. Conf. Learn. Represent.*, 2025, pp. 1–41.

[235] C. Wang et al., "Efficient training of multi-task neural solver for combinatorial optimization," *Trans. Mach. Learn. Res.*, pp. 1–24, 2025.

[236] Y. L. Goh et al., "SHIELD: Multi-task multi-distribution vehicle routing solver with sparsity & hierarchy in efficiently layered decoder," in *Proc. Int. Conf. Mach. Learn.*, 2025, pp. 19679–19711.

[237] F. Berto et al., "RL4CO: An extensive reinforcement learning for combinatorial optimization benchmark," in *Proc. 31st ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2025, pp. 5278–5289.

[238] A. Noureddine, "PowerJoular and JoularJX: Multi-platform software power monitoring tools," in *Proc. 18th Int. Conf. Intell. Environments (IE)*, Jun. 2022, pp. 1–5.

[239] H. Lu et al., "ROCO: A general framework for evaluating robustness of combinatorial optimization solvers on graphs," in *Proc. Int. Conf. Learn. Represent.*, 2023, pp. 1–23.

[240] L. Li et al., "An end-to-end OCR text re-organization sequence learning for rich-text detail image comprehension," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 85–100.

[241] Y. Tang et al., "Multi-modal segment assemblage network for ad video editing with importance-coherence reward," in *Proc. Asian Conf. Comput. Vis.*, 2022, pp. 3519–3535.

[242] Z. Zhang, X. Wang, Z. Zhang, P. Cui, and W. Zhu, "Revisiting transformation invariant geometric deep learning: An initial representation perspective," 2021, *arXiv:2112.12345*.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:48:06 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.



Xuan Wu received the B.Sc. degree from the College of Computer Science and Technology, Jilin University, Changchun, China, in 2020. He is currently pursuing the Ph.D. degree with Jilin University.

From November 2023 to November 2024, he was a Visiting Student at the College of Computing and Data Science, Nanyang Technological University, Singapore. He has published more than 20 journal articles and conference papers. His main research interests include neural combinatorial optimization, neural architecture search, and large language models.



Lijie Wen received the B.S., M.S., and Ph.D. degrees from the Department of Computer Science and Technology, Tsinghua University, Beijing, China, in 2000, 2003, and 2007, respectively.

He is currently a tenure Associate Professor at the School of Software, Tsinghua University. He has published more than 270 academic papers in conferences and journals, which have been cited more than 8000 times by Google Scholar. His research interests include natural language processing, large language models, process data management, and lifecycle management of computational workflow.



Yubin Xiao received the M.Sc. degree from Xiangtan University, Xiangtan, China, in 2021, and the Ph.D. degree from Jilin University, Changchun, China, in 2025.

His main research interests include neural combinatorial optimization and machine learning.



Chunguo Wu received the B.S. and M.S. degrees from the Department of Mathematics, Jilin University, Changchun, China, in 1999 and 2002, respectively, and the Ph.D. degree from the College of Computer Science and Technology, Jilin University, in 2006.

He is currently a Professor at the College of Computer Science and Technology, Jilin University. He has authored two books, holds two invention patents, and has published over 100 papers in venues such as SIGKDD, IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, IEEE TRANSACTIONS ON MULTIMODAL COMPUTING AND INTERACTION, and IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION. His research interests include evolutionary computation and adaptive systems.



Yuesong Wu received the B.Sc. degree from Shandong University of Science and Technology, Qingdao, China, in 2023, and the M.Sc. degree from Jilin University, Changchun, Jilin, China, in 2026.

His main research interests include neural combinatorial optimization and machine learning.



Chaoyu Yu received the B.Sc. and M.Sc. degrees from Jilin University, Changchun, Jilin, China, in 2022 and 2025, respectively.

His main research interests include neural combinatorial optimization.



Douglas L. Maskell (Life Senior Member, IEEE) received the B.E. (Hons.), M.Eng.Sc., and Ph.D. degrees in electronic and computer engineering from James Cook University, Douglas, QLD, Australia, in 1980, 1984, and 1996, respectively.

He is currently an Associate Professor with the College of Computing and Data Science, Nanyang Technological University, Singapore. His research interests include embedded systems, reconfigurable computing, intelligent systems, and algorithm acceleration for hybrid high-performance computing systems.



You Zhou received the B.Sc. and Ph.D. degrees from Jilin University, Changchun, China, in 2002 and 2008, respectively.

He is currently a Professor with the College of Computer Science and Technology and the Vice Dean of the College of Software, Jilin University. He has published more than 200 journal articles and conference papers. His research interests include neural combinatorial optimization, pattern recognition, and machine learning.



Di Wang (Senior Member, IEEE) received the B.Eng. and Ph.D. degrees in computer science from Nanyang Technological University (NTU), Singapore, in 2003 and 2014, respectively.

He is currently a Senior Research Fellow and the Research Manager with the Joint NTU-UBC Research Center of Excellence in Active Living for the Elderly, NTU. His research interests include AI for healthcare, affective computing, graph models, constraint optimizations, autonomous agents, and reasoning under uncertainty.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:48:06 UTC from IEEE Xplore. Restrictions apply.